

RTOS Book

Mohan Arunachalam

Contents

1	The Bare Machine	3
1.1	What Is an Embedded System?	3
1.2	Anatomy of a Microcontroller	3
1.3	The Memory Map	5
1.3.1	Code: Reading a peripheral register directly	5
1.4	The Clock System	6
1.5	The Reset Sequence — From Power-On to main()	8
1.5.1	The Vector Table	8
1.5.2	Code: Minimal startup file (C version)	9
1.6	SRAM Layout	11
1.6.1	Code: Verifying the layout in code	11
1.7	The Linker Script: Giving Memory its Shape	12
1.8	The Unsolved Tension	13
2	Interrupts — The Hardware’s Voice	14
2.1	The Problem With the Super-Loop	14
2.2	What is an Interrupt?	15
2.3	The ARM Cortex-M Interrupt Hardware	15
2.4	What the CPU Does When an Interrupt Fires	15
2.5	The Foreground / Background Model	16
2.6	Interrupt Nesting	17
2.7	Critical Sections — Protecting Shared Data	17
2.7.1	Rules for Critical Sections	18
2.8	The volatile Keyword Revisited	18
2.9	SysTick — The Heartbeat Timer	19
2.10	Putting It Together: UART + SysTick Without an RTOS	20
2.11	The Unsolved Tension	21
3	Tasks — Pretending to Multitask	22
3.1	The Core Idea: Illusion of Parallelism	22
3.2	What is a “Context”?	23
3.3	The Task Control Block (TCB)	23
3.3.1	Code: Minimal TCB definition	23
3.4	Task States and Transitions	24
3.5	The Task Stack	25
3.5.1	Code: Initialising a task stack	26
3.6	The Context Switch — Heart of the RTOS	27
3.6.1	Code: Context switch in assembly (ARM Cortex-M, GCC syntax)	28
3.7	Task Creation — Wiring It All Together	29
3.8	The Ready List — How the Scheduler Sees Tasks	32
3.9	The Unsolved Tension	32
4	Concepts and Recommendations	33
4.1	The Pitfalls of Dynamic Memory (malloc)	33
4.2	Task States and Timing Analysis	33
4.3	Queues: The Right Way to Pass Data	33

4.4	Software Timers vs. Hardware Timers	34
4.5	The Tick Hook & System Heartbeat	34
4.6	The Idle Task & Power Management	34
4.7	Event Groups (The “Wait for All” logic)	34
5	Problem solving	35
5.1	Odd / Even Print	35
5.2	The Producer-Consumer Pattern	35
5.3	Queue Overflow	36
5.4	Handling Shared Hardware (The Re-entrancy Problem)	37
5.5	The Deadlock Scenario	38
5.6	Priority Inversion:	39
5.7	Interrupt to Task Communication	39
6	Zephyr RTOS in Practice: Build Scalable Embedded Application	41
6.1	Introduction	41
6.1.1	What is Zephyr?	41
6.1.2	Where Zephyr Comes From?	41
6.1.3	Zephyr’s Key Characteristics	41
6.1.4	Why You Should Learn Zephyr	42
6.1.5	Zephyr from the Arduino Perspective	42
6.1.6	Why Arduino Users Should Learn Zephyr	42
6.1.7	Zephyr Architecture Overview	42
6.1.8	How Zephyr Projects Are Structured	43
6.1.9	An Introduction to Kconfig / prj.conf	44
6.1.10	An Introduction to Devicetree	44
6.1.11	West Fundamentals	46
6.1.12	Zephyr logging	46
6.2	Examples	47
6.2.1	Blinky	47

Chapter 1

The Bare Machine



Before a task can run, before a scheduler can tick, before an RTOS can breathe — a microcontroller must first wake up and answer one question: “What do I do next?” This chapter is the answer.

1.1 What Is an Embedded System?

An embedded system is a computer built for **one specific job**, permanently. It is not a general-purpose PC. It has no operating system you can “install”, no hard drive, no display (usually), and often no keyboard. It is a microcontroller soldered to a board, powered by a battery or a wall wart, and expected to do one thing — forever, reliably, with tight constraints on power, cost, and memory.

Examples you interact with every day:

Device	MCU Role
Washing machine	Controls drum motor, water valve, heater based on program
Car ABS system	Reads wheel speed sensors, pulses brakes 15× per second
Cardiac pacemaker	Monitors heart rhythm, delivers pulse if it detects absence
Smart thermostat	Reads temperature, drives HVAC relay, communicates over Wi-Fi
USB keyboard	Scans key matrix, builds HID report, sends over USB every 1 ms

All of these run on bare or near-bare metal. Understanding *how the machine wakes up* is the foundation for everything else in this book.

1.2 Anatomy of a Microcontroller

A microcontroller (MCU) is a **system-on-chip**: it integrates a CPU core, memory, and peripherals into one package. Unlike a microprocessor (which needs external RAM and ROM), the MCU has everything it needs on-die.

Key sub-components:

- **CPU Core** — The Cortex-M4 (or M0, M7, RISC-V, etc.) that fetches and executes instructions. Contains the Arithmetic Logic Unit (ALU), general-purpose registers (R0-R15 on ARM), and the **Program Counter** (PC).
- **Flash memory** — Non-volatile. Survives power-off. Your compiled program lives here. Read-only during normal execution (write requires special erase/program sequences).
- **SRAM** — Volatile. Loses content on power-off. Holds your stack, heap, and global variables at runtime.

- **Peripherals** – Hardware blocks that do work in parallel with the CPU: GPIO, UART, SPI, I2C, ADC, timers, DMA, and the Watchdog Timer (WDT).
- **Internal Bus** – The AHB/APB bus (on ARM Cortex-M) or similar fabric that lets the CPU read/write memory and peripheral registers using the same address-space mechanism.

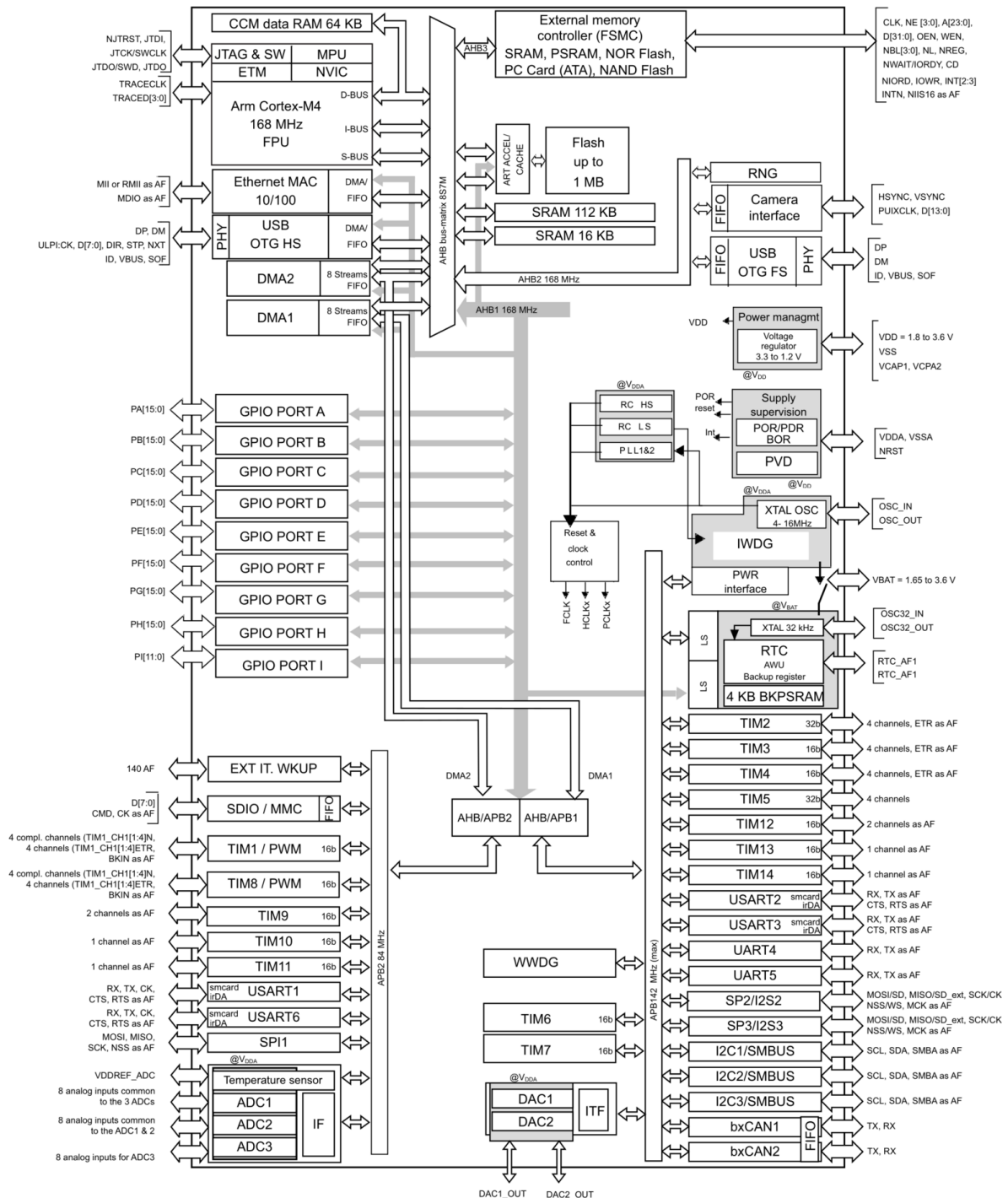


Figure 1.1: STM32 MCU block diagram

1.3 The Memory Map

When the CPU wants to read a byte, it puts an **address** on the bus. The bus fabric routes that address to the right destination — Flash, SRAM, or a peripheral register. This routing table is called the **memory map**.

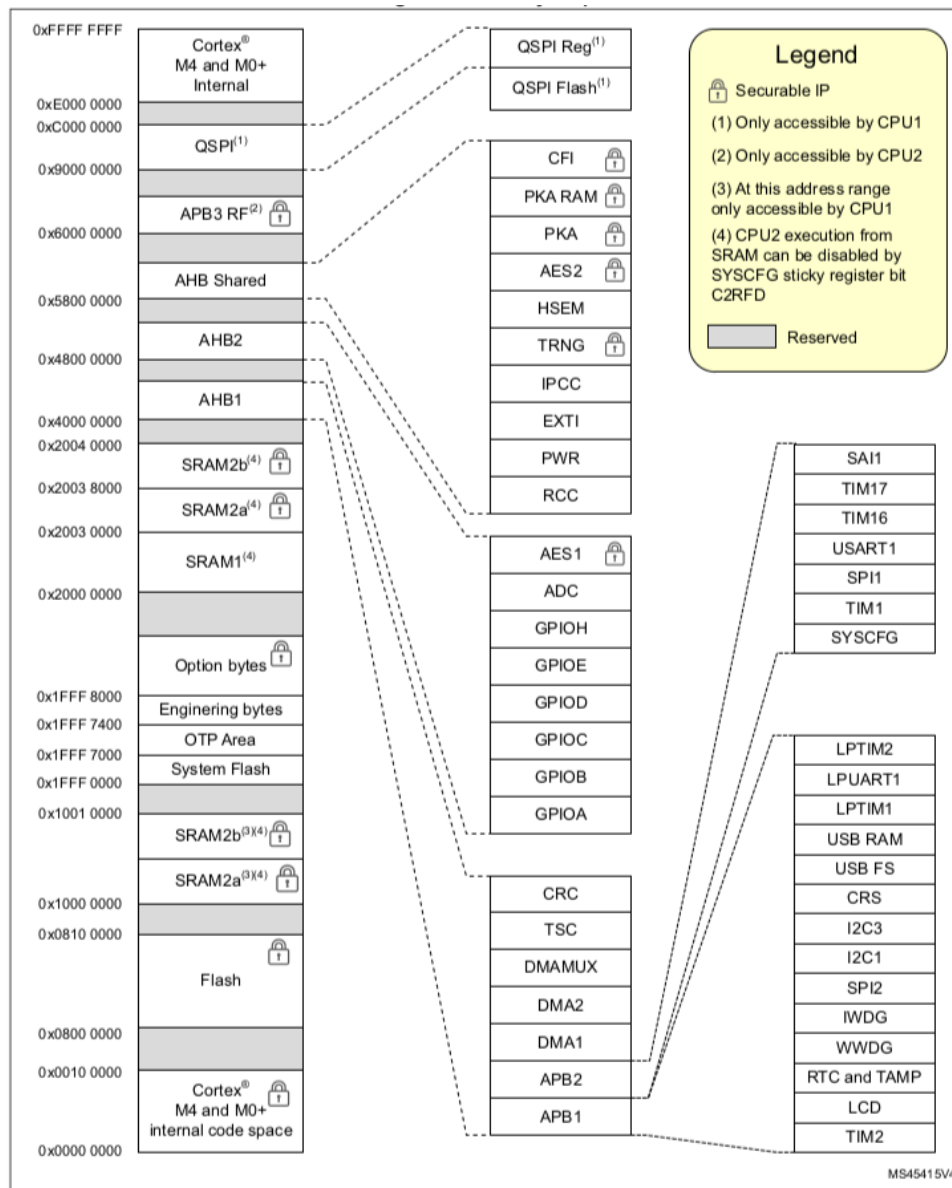


Figure 1.2: Memory Map

Why this matters for RTOS: When an RTOS writes to a GPIO pin, enables a timer, or pends a software interrupt — it is *simply writing a value to an address*. Understanding the memory map demystifies every line of peripheral driver code you will ever read.

1.3.1 Code: Reading a peripheral register directly

```
1 #include <stdint.h>
2
3 /* Peripheral base addresses (from the datasheet) */
4 #define PERIPH_BASE      0x40000000UL
5 #define APB2PERIPH_BASE (PERIPH_BASE + 0x00010000UL)
6 #define GPIOA_BASE      (APB2PERIPH_BASE + 0x00000000UL)
```

```

7
8  /* GPIOA register layout (simplified STM32F1) */
9  typedef struct {
10     volatile uint32_t CRL; /* Port config low */
11     volatile uint32_t CRH; /* Port config high */
12     volatile uint32_t IDR; /* Input data */
13     volatile uint32_t ODR; /* Output data */
14     volatile uint32_t BSRR; /* Bit set/reset */
15     volatile uint32_t BRR; /* Bit reset */
16     volatile uint32_t LCKR; /* Port lock */
17 } GPIO_TypeDef;
18
19 #define GPIOA ((GPIO_TypeDef *)GPIOA_BASE)
20
21 /* Toggle PA5 (LED on Nucleo board) */
22 void toggle_led(void) {
23     GPIOA->ODR ^= (1u << 5); /* Read-Modify-Write on ODR */
24 }

```



Note the `volatile` keyword. Without it, the compiler might cache the register value in a CPU register and never re-read it from hardware. `volatile` tells the compiler: “this memory location can change outside your knowledge — always go to the actual address.”

1.4 The Clock System

A microcontroller is a **synchronous digital system**: every flip-flop changes state on a clock edge. Without a clock signal, nothing moves. The clock frequency determines how many instructions per second the CPU can execute.

Typical clock initialization (bare-metal, C):

```

1  /*
2   * clock_init.c
3   *
4   * Configure STM32F4 to run at 168 MHz using HSE (8 MHz crystal) + PLL.
5   * This is often the first thing called from Reset_Handler.
6   */
7
8  #include "stm32f4xx.h" /* CMSIS device header – defines RCC, FLASH, etc. */
9
10 void clock_init(void)
11 {
12     /* 1. Enable HSE oscillator and wait until it is stable */
13     RCC->CR |= RCC_CR_HSEON;
14     while (!(RCC->CR & RCC_CR_HSERDY)); /* spin until hardware sets HSERDY */
15
16     /* 2. Configure Flash wait states for 168 MHz (5 wait states, ART on) */
17     FLASH->ACR = FLASH_ACR_LATENCY_5WS
18                 | FLASH_ACR_PRFTEN
19                 | FLASH_ACR_ICEN
20                 | FLASH_ACR_DCEN;
21
22     /* 3. Set PLL source = HSE, configure M/N/P/Q dividers
23      *   SYSCLK = (HSE / M) * N / P = (8 / 8) * 336 / 2 = 168 MHz
24      *   USB FS clock = (HSE / M) * N / Q = 1 * 336 / 7 = 48 MHz */

```

```

25  RCC->PLLCFGR = (8u << RCC_PLLCFGR_PLLM_Pos)    /* M = 8 */
26                | (336u << RCC_PLLCFGR_PLLN_Pos)   /* N = 336 */
27                | (0u << RCC_PLLCFGR_PLLP_Pos)     /* P = 2 (00b) */
28                | (7u << RCC_PLLCFGR_PLLQ_Pos)     /* Q = 7 */
29                | RCC_PLLCFGR_PLLSRC_HSE;          /* source = HSE */
30
31  /* 4. Enable PLL and wait for lock */
32  RCC->CR |= RCC_CR_PLLON;
33  while (!(RCC->CR & RCC_CR_PLLRDY));
34
35  /* 5. Set AHB/APB prescalers:
36   *   AHB  = SYSCLK / 1 = 168 MHz
37   *   APB1 = SYSCLK / 4 = 42 MHz (max 42 MHz for APB1)
38   *   APB2 = SYSCLK / 2 = 84 MHz (max 84 MHz for APB2) */
39  RCC->CFGR = RCC_CFGR_HPRE_DIV1
40              | RCC_CFGR_PPRE1_DIV4
41              | RCC_CFGR_PPRE2_DIV2;
42
43  /* 6. Switch SYSCLK source to PLL */
44  RCC->CFGR |= RCC_CFGR_SW_PLL;
45  while ((RCC->CFGR & RCC_CFGR_SWS) != RCC_CFGR_SWS_PLL); /* confirm switch */
46
47  /* Done - CPU now runs at 168 MHz */
48  }

```

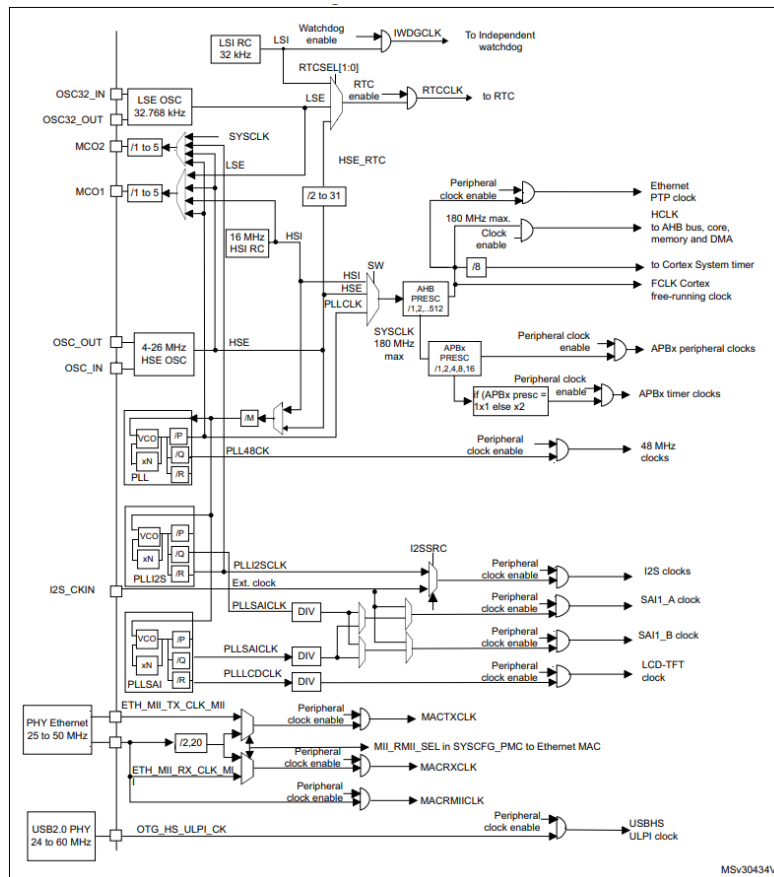


Figure 1.3: Clock Tree



Key insight: Every clock configuration is a sequence of register writes. The hardware handshake (`while(!(RCC->CR & flag))`) is critical — switching to an unstable clock is a common source of hard faults in embedded bring-up.

1.5 The Reset Sequence — From Power-On to `main()`

This is the most important section of the chapter. When power is applied, the MCU does not jump to `main()` . It goes through a precise, hardware-defined boot sequence.

```
1 Power Applied
2   V
3 +-----+
4 | Step 1: Hardware Reset                               |
5 | • All registers reset to default values              |
6 | • CPU reads address 0x0000_0000: Initial Stack Pointer (MSP) |
7 | • CPU reads address 0x0000_0004: Reset Handler address (PC) |
8 +-----+
9           V
10 +-----+
11 | Step 2: Reset_Handler (startup assembly)            |
12 | • Set up Stack Pointer (already done by hardware from vector) |
13 | • Copy .data section from Flash -> SRAM              |
14 | • Zero-fill .bss section in SRAM                    |
15 +-----+
16           V
17 +-----+
18 | Step 3: main()                                       |
19 +-----+
```

1.5.1 The Vector Table

The **vector table** is an array of 32-bit addresses stored at the very beginning of Flash. The CPU's very first act after reset is to read two entries from this table:

Memory Address		Exception Number
0x0000004C	Interrupt#3 vector	19
0x00000048	Interrupt#2 vector	18
0x00000044	Interrupt#1 vector	17
0x00000040	Interrupt#0 vector	16
0x0000003C	SysTick vector	15
0x00000038	PendSV vector	14
0x00000034	Not used	13
0x00000030	Not used	12
0x0000002C	SVC vector	11
0x00000028	Not used	10
0x00000024	Not used	9
0x00000020	Not used	8
0x0000001C	Not used	7
0x00000018	Not used	6
0x00000014	Not used	5
0x00000010	Not used	4
0x0000000C	Hard Fault vector	3
0x00000008	NMI vector	2
0x00000004	Reset vector	1
0x00000000	MSP initial value	0

Note: LSB of each vector must be set to 1 to indicate Thumb state

Figure 1.4: Vector Table - ARM Cortex-M

1.5.2 Code: Minimal startup file (C version)

```
1  /*
2   * startup.c – Minimal bare-metal startup for ARM Cortex-M
3   *
4   * Normally this is written in assembly (startup_stm32xxx.s) by the
5   * vendor. This C version makes the logic transparent.
6   */
7
8  #include <stdint.h>
9  #include <string.h>  /* memset, memcpy */
10
11 /* Linker script symbols – defined in the .ld file, not in C */
12 extern uint32_t _sidata; /* start of .data in Flash (load address) */
13 extern uint32_t _sdata;  /* start of .data in SRAM */
14 extern uint32_t _edata;  /* end   of .data in SRAM */
15 extern uint32_t _sbss;   /* start of .bss in SRAM */
16 extern uint32_t _ebss;   /* end   of .bss in SRAM */
17 extern uint32_t _estack; /* initial stack pointer (top of SRAM) */
18
19 /* Forward declarations */
20 int main(void);
21 void SystemInit(void);
22
23 /* Default handler – loops forever, easy to catch in a debugger */
24 void Default_Handler(void) { while (1); }
25
26 /* Weak aliases – peripherals override these in their own files */
27 void NMI_Handler(void)      __attribute__((weak, alias("Default_Handler")));
28 void HardFault_Handler(void) __attribute__((weak, alias("Default_Handler")));
29 void SysTick_Handler(void)  __attribute__((weak, alias("Default_Handler")));
30 /* ... more weak aliases for each IRQ ... */
31
32 /*
33  * Reset_Handler – entry point after every reset.
34  * Placed in .text, called by hardware via the vector table.
35  */
36 void Reset_Handler(void)
37 {
38     uint32_t *src, *dst;
39
40     /*
41      * 1. Copy initialised globals/statics from Flash to SRAM.
42      * The linker script places the initial values after .text in Flash.
43      * At runtime they must live in SRAM so they are writable.
44      *
45      * Example: uint32_t x = 42; -> value 42 lives in Flash,
46      * but x's address is in SRAM. We copy it here.
47      */
48     src = &_sidata;          /* source: Flash load address */
49     dst = &_sdata;          /* destination: SRAM VMA */
50     while (dst < _edata) {
51         *dst++ = *src++;
52     }
53
54     /*
55      * 2. Zero-initialise the .bss section.
56      * C standard guarantees zero-init for globals with no initialiser.
57      */
58 }
```

```

57     *   Example: static int counter; -> must be 0 at startup.
58     */
59     dst = &_sbss;
60     while (dst < &_ebss) {
61         *dst++ = 0u;
62     }
63
64     /*
65     * 3. Optional: initialise clocks, FPU, MPU (chip-specific).
66     *   Many CMSIS/HAL libraries provide SystemInit() for this.
67     */
68     SystemInit();
69
70     /*
71     * 4. Call main(). If main() returns (it shouldn't), spin.
72     */
73     main();
74     while (1);    /* safety net */
75 }
76
77 /*
78 * Vector table - placed in a special linker section ".isr_vector"
79 * which the linker script maps to address 0x0800_0000.
80 *
81 * Entry [0] = Initial MSP value (top of stack)
82 * Entry [1] = Reset_Handler address
83 * Remaining = IRQ handler addresses
84 */
85 __attribute__((section(".isr_vector"))) const uint32_t vector_table[] = {
86     (uint32_t)&_estack,          /* [0] Initial MSP          */
87     (uint32_t)Reset_Handler,     /* [1] Reset                */
88     (uint32_t)NMI_Handler,       /* [2] NMI                  */
89     (uint32_t)HardFault_Handler, /* [3] Hard fault          */
90     0, 0, 0, 0,                 /* [4-7] reserved          */
91     0, 0, 0,                   /* [8-10] reserved         */
92     (uint32_t)Default_Handler,   /* [11] SVCall             */
93     0, 0,                      /* [12-13] reserved        */
94     (uint32_t)Default_Handler,   /* [14] PendSV <- RTOS uses this */
95     (uint32_t)SysTick_Handler,   /* [15] SysTick <- RTOS tick    */
96     /* Device-specific IRQ handlers follow ... */
97 };

```

1.6 SRAM Layout

Once `Reset_Handler` finishes, SRAM is organised into distinct regions. Understanding this layout prevents the most common class of embedded bugs: stack overflow and heap corruption.

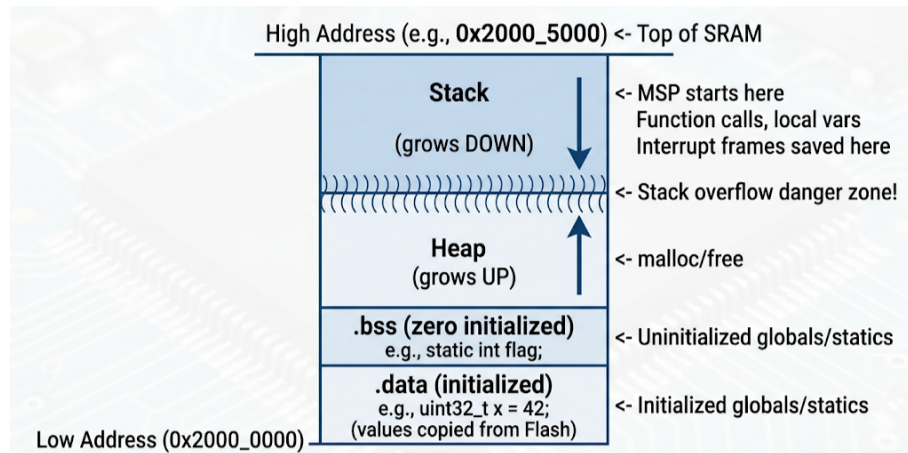


Figure 1.5: SRAM layout

1.6.1 Code: Verifying the layout in code

```

1  /*
2   * memory_layout.c – Print the addresses of key memory regions.
3   * Useful during bring-up to confirm the linker script is correct.
4   * Compile for host (PC) or embed in a UART debug build.
5   */
6
7  #include <stdio.h>
8  #include <stdint.h>
9
10 /* A zero-initialised global (goes in .bss) */
11 static uint32_t bss_example;
12
13 /* An initialised global (goes in .data, initial value in Flash) */
14 static uint32_t data_example = 0xDEADBEEF;
15
16 /* A constant (goes in Flash .rodata – not in SRAM) */
17 static const uint32_t rodata_example = 0xCAFEBAFE;
18
19 void print_memory_layout(void)
20 {
21     uint32_t stack_local; /* local variable – lives on the stack */
22
23     printf("=== Memory Layout ===\n");
24     printf(".rodata (Flash) : %p value=0x%08X\n",
25           (void *)&rodata_example, rodata_example);
26     printf(".data (SRAM) : %p value=0x%08X\n",
27           (void *)&data_example, data_example);
28     printf(".bss (SRAM) : %p value=0x%08X\n",
29           (void *)&bss_example, bss_example);
30     printf("stack local var : %p\n",
31           (void *)&stack_local);
32
33     /* On a real MCU you'd see:

```



```

34     * .rodata: 0x0800xxxx (in Flash)
35     * .data:   0x2000xxxx (in SRAM)
36     * .bss:    0x2000yyyy (just above .data)
37     * stack:   0x2000zzzz (near top of SRAM) */
38 }

```

1.7 The Linker Script: Giving Memory its Shape

The linker script (`.ld` file) is the blueprint that tells the linker where to place each section in the final binary. It directly controls the SRAM layout described above.

```

1  /* minimal.ld - Simplified GNU linker script for STM32F401 */
2
3  MEMORY
4  {
5      /* Flash: 256 KB starting at 0x0800_0000 */
6      FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 256K
7
8      /* SRAM: 64 KB starting at 0x2000_0000 */
9      SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
10 }
11
12 /* Initial stack pointer: top of SRAM */
13 _estack = ORIGIN(SRAM) + LENGTH(SRAM); /* = 0x20010000 */
14
15 SECTIONS
16 {
17     /* --- Flash sections --- */
18
19     .isr_vector : /* Vector table must be at Flash start */
20     {
21         KEEP(*(.isr_vector))
22     } > FLASH
23
24     .text : /* Program code and read-only data */
25     {
26         *(.text)
27         *(.text*)
28         *(.rodata)
29         *(.rodata*)
30         _sidata = .; /* Mark: initial values for .data follow */
31     } > FLASH
32
33     /* --- SRAM sections --- */
34
35     .data : AT(_sidata) /* Load from Flash (AT), run in SRAM */
36     {
37         _sdata = .; /* Start of .data in SRAM */
38         *(.data)
39         *(.data*)
40         _edata = .; /* End of .data in SRAM */
41     } > SRAM
42
43     .bss :
44     {
45         _sbss = .; /* Start of .bss */
46         *(.bss)

```

```

47     *(.bss*)
48     *(COMMON)
49     _ebss = .;           /* End of .bss */
50 } > SRAM
51
52 /* Heap starts at _ebss, stack at _estack – they must not meet! */
53 }

```



Linker script = memory contract. When you see a HardFault caused by an illegal address, the linker script is almost always where you should look first. Verify that your Flash and SRAM sizes exactly match the datasheet.

1.8 The Unsolved Tension

The LED blinks. The machine is alive. But look at what `main()` does:

```

1 while (1)
2 {
3     do_thing_A(); /* blink LED */
4     delay_cycles(1000000u);
5     do_thing_B(); /* ??? */
6 }

```

This loop is sequential. `do_thing_B()` cannot run while `do_thing_A()` is waiting. If you need to simultaneously blink an LED every 500 ms, read a UART byte every 1 ms, and update an LCD every 100 ms — this single loop cannot do it cleanly.

The solution is **concurrency**. But before we reach for an RTOS, we first need to understand the simplest form of concurrency the hardware gives us for free: **interrupts**. That is the subject of Chapter 2.

End of Chapter 1. In Chapter 2, we will see how the hardware's interrupt mechanism lets us escape the tyranny of the sequential loop — and why it creates its own new set of problems.

Chapter 2

Interrupts — The Hardware’s Voice



In Chapter 1 the machine booted and ran a single loop. It worked — until we needed two things to happen at once. The hardware solves this with a mechanism that has existed since the 1950s: the interrupt. Understanding it deeply is not optional. Every RTOS is built on top of it.

2.1 The Problem With the Super-Loop

End of Chapter 1 left us with this pattern:

```
1 int main(void)
2 {
3     while (1)
4     {
5         blink_led();           /* 500 ms blocking delay inside */
6         read_uart();           /* needs to run every 1 ms      */
7         update_lcd();          /* needs to run every 100 ms   */
8     }
9 }
```

The loop is **sequential and blocking**. While `blink_led()` sits in its half-second delay, `read_uart()` cannot run. UART bytes arrive at 115200 baud — one byte every $\sim 87\ \mu\text{s}$. The super-loop misses them all.

This is the **timing problem**: a single thread of execution cannot be in two places simultaneously.

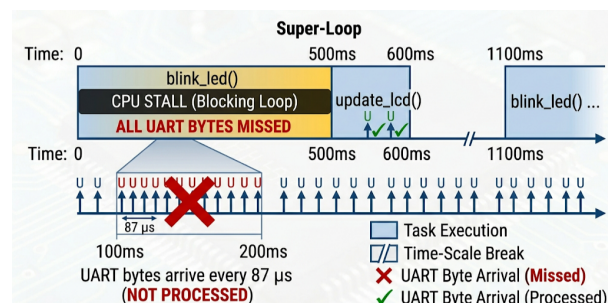


Figure 2.1: Timeline of Superloop

The hardware’s answer to this is the **interrupt**: a signal that tells the CPU “stop what you are doing, handle me, then come back.”

2.2 What is an Interrupt?

An interrupt is a hardware signal that causes the CPU to **suspend its current execution**, save its state, and jump to a special function called an **Interrupt Service Routine (ISR)**. When the ISR returns, the CPU restores its state and resumes exactly where it left off.

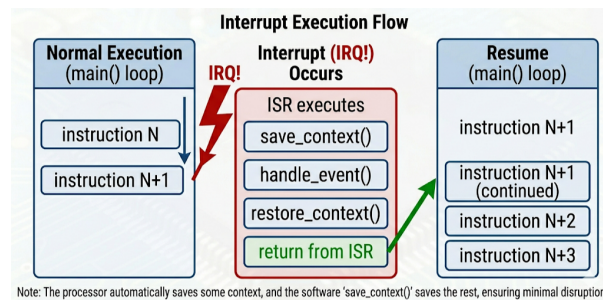


Figure 2.2: ISR execution flow

Key points:

1. The CPU does NOT finish the current instruction block – it interrupts it
2. The return address is saved so execution resumes at exactly the right spot
3. The ISR must be short – the longer it runs, the more main() is delayed

This mechanism is implemented entirely in hardware – the CPU itself detects the interrupt signal, pushes a stack frame, and vectors to the ISR address from the vector table.

2.3 The ARM Cortex-M Interrupt Hardware

On ARM Cortex-M, interrupts are managed by the **Nested Vectored Interrupt Controller (NVIC)**. It is the traffic cop between peripheral signals and the CPU.

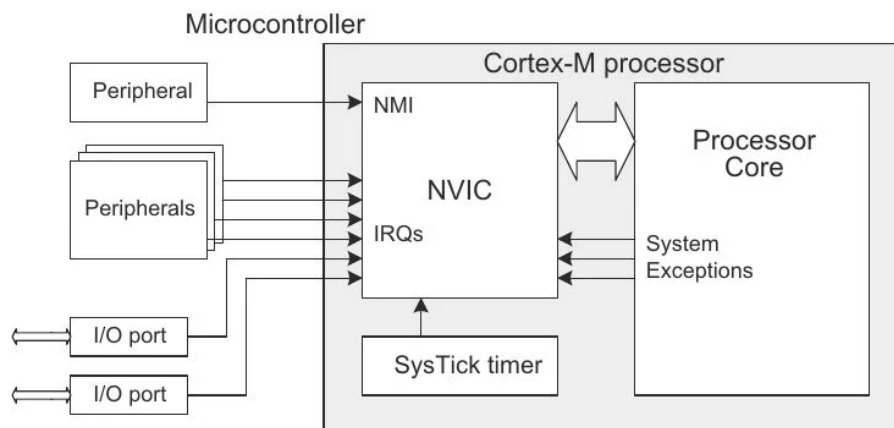


Figure 2.3: Interrupt controller

Priority: Lower number = higher priority.

On Cortex-M4 with 4 priority bits, valid priorities are 0 (highest) through 15 (lowest). The RTOS reserves the lowest priorities for its own PendSV and SysTick handlers.

2.4 What the CPU Does When an Interrupt Fires

When an interrupt arrives and its priority is higher than the currently executing code, the CPU performs an **automatic context save** (called “exception entry”) in hardware:

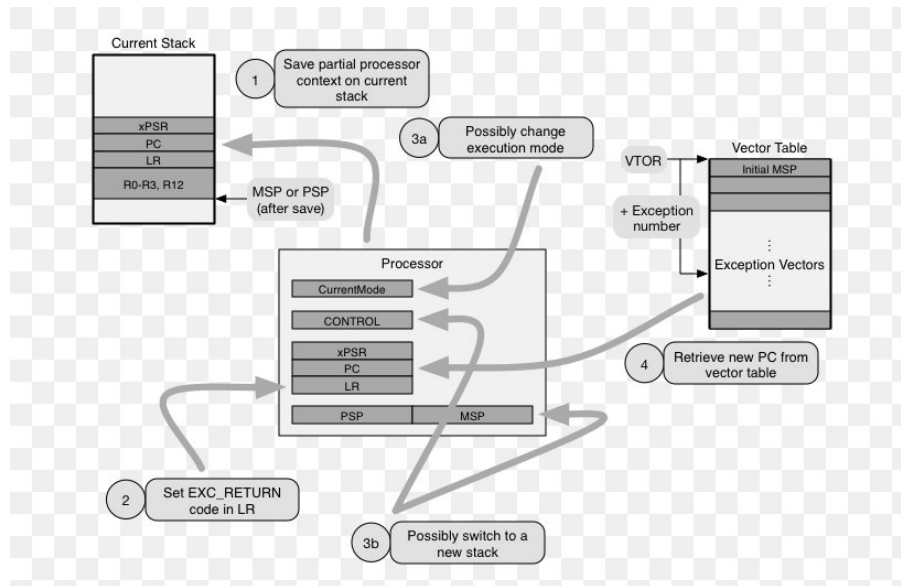


Figure 2.4: ARM Cortex-M Exception Entry

- After ISR stack frame push, CPU:
 1. Sets PC = ISR address (from vector table)
 2. Sets LR = EXC_RETURN (magic value to trigger unstacking on return)
 3. Executes ISR
- On ISR return (BX LR with EXC_RETURN value):
 1. Hardware pops R0–R3, R12, LR, PC, xPSR from stack
 2. SP restored to original value
 3. CPU resumes interrupted instruction stream

This hardware-assisted save/restore is fast (~12 cycles on Cortex-M4) and is what makes ISRs practical for tight-timing applications. The RTOS context switch mechanism.

2.5 The Foreground / Background Model

Once interrupts are in play, the system has two levels of execution:

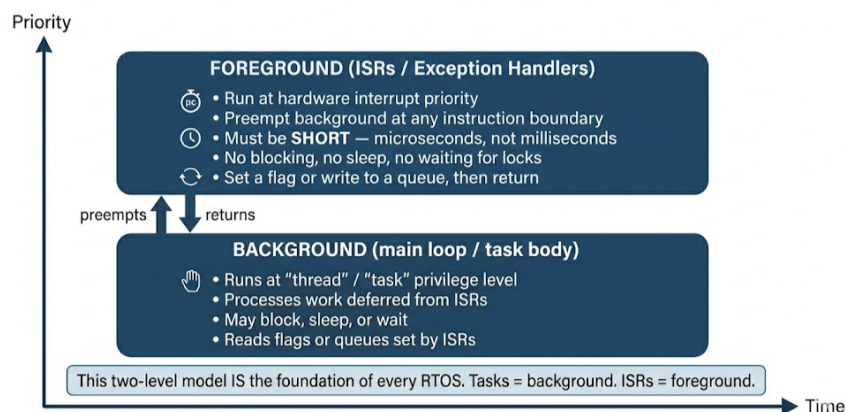


Figure 2.5: Two level Model

The ISR's job is simple: **detect the event, record it, return**. The heavy processing happens in the background where blocking is safe.

2.6 Interrupt Nesting

On ARM Cortex-M, a higher-priority interrupt can preempt a lower-priority ISR that is already running. This is **interrupt nesting**.

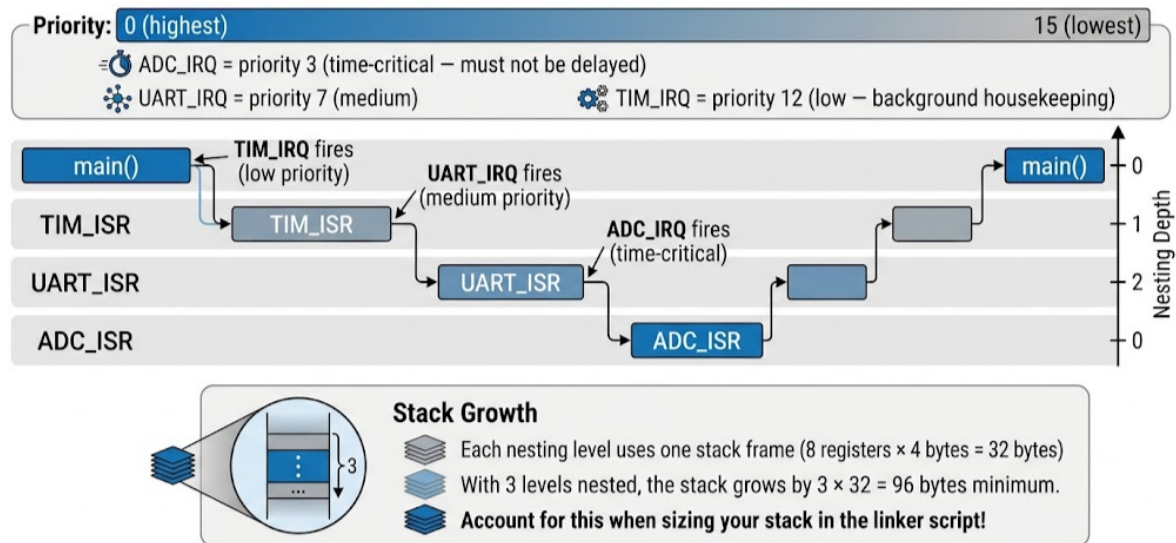


Figure 2.6: Nested Interrupt

2.7 Critical Sections — Protecting Shared Data

The ISR and the background loop share data. Most of the time this is safe, but some operations are **non-atomic** — they require multiple instructions to complete, and an interrupt in the middle leaves data in an inconsistent state.

The fix is a **critical section**: a region of code that cannot be interrupted.

```

1  /*
2   * critical_section.c - Protecting shared state on ARM Cortex-M
3   */
4
5  #include <stdint.h>
6  #include "cmsis_compiler.h" /* __get_PRIMASK, __set_PRIMASK, etc. */
7
8  /*
9   * Enter critical section: disable all maskable interrupts.
10   * Returns the previous PRIMASK value so it can be restored.
11   *
12   * PRIMASK = 1 -> all IRQs (except NMI and HardFault) are masked.
13   */
14  static inline uint32_t critical_enter(void)
15  {
16      uint32_t primask = __get_PRIMASK();
17      __disable_irq(); /* sets PRIMASK = 1 (CPSID I) */
18      __DSB();          /* data sync barrier */
19      __ISB();          /* instruction sync barrier */
20      return primask;
21  }
22
23  /*

```

```

24  * Exit critical section: restore interrupts to previous state.
25  * Using the saved PRIMASK rather than unconditionally enabling
26  * handles nested critical sections correctly.
27  */
28  static inline void critical_exit(uint32_t primask)
29  {
30      __set_PRIMASK(primask);    /* restore saved state          */
31  }
32
33  /* -- Usage example -- */
34
35  static volatile uint32_t shared_counter = 0;
36
37  void increment_counter_safely(void)
38  {
39      uint32_t saved = critical_enter();
40      {
41          shared_counter++;      /* now atomic - ISR cannot fire */
42      }
43      critical_exit(saved);
44  }
45
46  /*
47   * FreeRTOS equivalent:
48   *   taskENTER_CRITICAL();
49   *   shared_counter++;
50   *   taskEXIT_CRITICAL();
51   *
52   * FreeRTOS uses BASEPRI (not PRIMASK) to mask only IRQs at or below
53   * configMAX_SYSCALL_INTERRUPT_PRIORITY, leaving high-priority ISRs
54   * running.
55   */

```

2.7.1 Rules for Critical Sections

- DO:
 - Keep them as SHORT as possible (nanoseconds to low microseconds)
 - Protect any multi-instruction read-modify-write on shared data
 - Save and restore PRIMASK (not unconditional enable/disable)
 - Use for flag polling between ISR and task
- DO NOT:
 - Call any blocking function inside (no sleep, no mutex wait)
 - Call printf, malloc, or any function with locks inside
 - Nest them without saving state (PRIMASK save/restore handles this)
 - Assume the compiler will make your operation atomic (even a++ is not atomic without the critical section)

2.8 The volatile Keyword Revisited

Every variable shared between an ISR and background code **must** be declared `volatile`. Without it, the compiler is free to optimise the read away:

```

1  /*
2   * volatile_demo.c - Why volatile is not optional for ISR-shared data
3   */
4
5  /* -- WRONG - compiler will optimise the loop away -- */
6  static uint8_t flag_wrong = 0;

```

```

7
8 void wait_for_event_WRONG(void)
9 {
10     /* Compiler sees: flag_wrong is never written in this function.
11      * Optimises this to: if (flag_wrong == 0) { while(1); }
12      * The ISR sets flag_wrong = 1, but the CPU reads a cached register.
13      * This loop NEVER exits even when the ISR fires. */
14     while (flag_wrong == 0)
15     {
16         /* spin */
17     }
18 }
19
20 /* -- CORRECT -- volatile forces a memory read every iteration -- */
21 static volatile uint8_t flag_correct = 0;
22
23 void wait_for_event_CORRECT(void)
24 {
25     /* Compiler cannot cache this - must read from memory each iteration.
26      * When ISR sets flag_correct = 1, this loop sees it immediately. */
27     while (flag_correct == 0)
28     {
29         /* spin */
30     }
31 }
32
33 /* ISR that sets the flag */
34 void EXTI0_IRQHandler(void)
35 {
36     flag_correct = 1;    /* volatile write - compiler generates STR instruction */
37
38     /* Clear the EXTI pending bit (required or IRQ will fire again) */
39     EXTI->PR = (1u << 0);
40 }

```

2.9 SysTick — The Heartbeat Timer

One special interrupt deserves extra attention: **SysTick**. It is a 24-bit countdown timer built into the Cortex-M core itself (not a peripheral — it lives in the system control space at 0xE000E000). It fires at a regular interval and is the timekeeping foundation of every RTOS.

SysTick Operation

- SysTick counts DOWN from RELOAD to 0, then fires, then reloads.
-

$$RELOADvalue = \frac{CPU_FREQ}{TICKRATE_HZ} - 1$$

- Example: 168 MHz CPU, 1000 Hz tick (1 ms period):
-

$$RELOAD = \frac{168000000}{1000} - 1 = 167999$$

Timeline

```

1 Count: 167999 -----> 0 | 167999 -----> 0 |
2                               |                               |
3 SysTick_Handler fires:      ^                               ^
4                               |                               |
5 Period: <----- 1 ms ----->|<----- 1 ms ----->|

```


- In a bare-metal system, SysTick_Handler increments a tick counter:

```
1 volatile uint32_t system_tick_ms = 0;
2 void SysTick_Handler(void) { system_tick_ms++; }
```

- In an RTOS, SysTick_Handler calls the scheduler — it is the heartbeat that drives task switching.

2.10 Putting It Together: UART + SysTick Without an RTOS

Here is the complete bare-metal application combining everything from this chapter. It echoes received UART bytes while blinking an LED at exactly 1 Hz — both happening “simultaneously.”

```
1  /*
2   * main.c - Foreground/background application with SysTick + UART ISR
3   *
4   * Two independent activities with no RTOS:
5   *   1. LED blinks every 500 ms (driven by SysTick tick counter)
6   *   2. UART echoes bytes (driven by UART RXNE interrupt)
7   *
8   * This works for 2 activities. For 10 activities, the code becomes
9   * an unmanageable nest of flags and timers – that is what an RTOS solves.
10  */
11
12  #include <stdint.h>
13  #include <stdbool.h>
14
15  extern void    systick_init(uint32_t);
16  extern uint32_t get_tick_ms(void);
17  extern void    uart1_init(uint32_t);
18  extern void    nvic_uart1_enable(void);
19  extern bool    uart1_read_byte(uint8_t *);
20  extern void    uart1_send_byte(uint8_t);
21  extern void    led_init(void);
22  extern void    led_toggle(void);
23
24  int main(void)
25  {
26      /* Hardware initialisation */
27      systick_init(84000000u);
28      uart1_init(84000000u);
29      nvic_uart1_enable();
30      led_init();
31
32      __enable_irq(); /* global interrupt enable */
33
34      uint32_t last_blink_ms = 0;
35      uint8_t byte;
36
37      while (1)
38      {
39          /* -- Activity 1: LED toggle every 500 ms -----
40           * Non-blocking: check elapsed time, do not spin.
41           * The ISRs can fire at any time while we are in this loop. */
42          uint32_t now = get_tick_ms();
43          if ((now - last_blink_ms) >= 500u)
44          {
45              led_toggle();
46              last_blink_ms = now;
47          }
```

```

48
49     /* -- Activity 2: UART echo -----
50     * Drain whatever bytes the UART ISR has deposited. */
51     while (uart1_read_byte(&byte))
52     {
53         uart1_send_byte(byte);
54     }
55
56     /* -- Future activities would be added here -----
57     * Activity 3: update LCD every 100 ms
58     * Activity 4: read ADC every 10 ms
59     * Activity 5: communicate over SPI
60     * ...
61     * This pattern does not scale. Every new activity adds more
62     * timing logic. Activities with blocking I/O stall others.
63     */
64 }
65 }

```

2.11 The Unsolved Tension

The foreground/background model works — for a small number of activities. But it has hard limits:

Limits of the Foreground/Background Model

- **Problem 1: No isolation:** All activities share one stack. A bug in Activity 3 corrupts Activity 1’s local variables. There is no protection boundary.
- **Problem 2: No priority between background activities:** Activity 1 (LED, low urgency) and Activity 4 (ADC, high urgency) are equals in the while(1) loop. You cannot say “ADC always runs before LED.”
- **Problem 3: Blocking anywhere stalls everything:** If Activity 3 must wait 10 ms for an SPI transaction to finish, all other background activities are frozen for 10 ms.
- **Problem 4: Timing accuracy degrades with more activities:** Each activity’s “run every N ms” check gets less accurate as more activities are added, because each one steals time from the others.
- **Problem 5: State machine complexity explodes:** Each activity needs its own state machine instead of being written as straight-line sequential code. With 10 activities this becomes unreadable.
- **The solution:** give each activity its OWN stack and execution context. That is a **TASK**

The interrupt mechanism gave us concurrency between hardware events and software. But it did not give us concurrency *between software tasks*. To get that, we need to introduce the abstraction that makes an RTOS: **the task**.

End of Chapter 2. We now understand how the hardware gives us concurrent execution through interrupts, and why the foreground/background model breaks down beyond a handful of activities. In Chapter 3, we will formalise the concept of a “task” — its own stack, its own context, its own life cycle — before letting the scheduler drive them in Chapter 4.



Chapter 3 Preview — Tasks: Pretending to Multitask We will define the Task Control Block (TCB), understand what “context” means precisely, hand-write a context switch in assembly, and see how a microcontroller can maintain the illusion of running multiple programs simultaneously — all without any additional hardware.

Chapter 3

Tasks — Pretending to Multitask



Chapter 2 showed that interrupts give us concurrency between hardware and software. But background activities still share one stack and one execution context. The RTOS solution is elegant: give each activity its own private stack and pretend each one has its own CPU. This pretence is so convincing that the programmer writes each task as if it runs forever, alone. This chapter builds that illusion from scratch.

3.1 The Core Idea: Illusion of Parallelism

A single-core CPU can only execute one instruction at a time. Yet every RTOS gives you the feeling that ten tasks are running simultaneously. How?

By switching between them so fast that the human observer cannot tell they are taking turns.

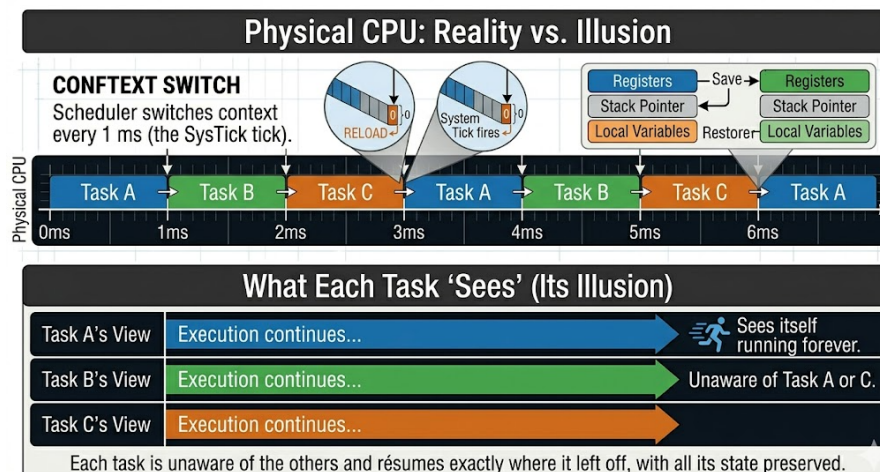


Figure 3.1: Task execution

To pull off this illusion, the RTOS needs two things:

1. A structure that describes each task and stores its saved state — the **Task Control Block (TCB)**.
2. A routine that can swap one task's state for another's — the **context switch**.

3.2 What is a “Context”?

A task’s **context** is everything the CPU needs to resume executing that task as if it had never been interrupted. On ARM Cortex-M, that is:

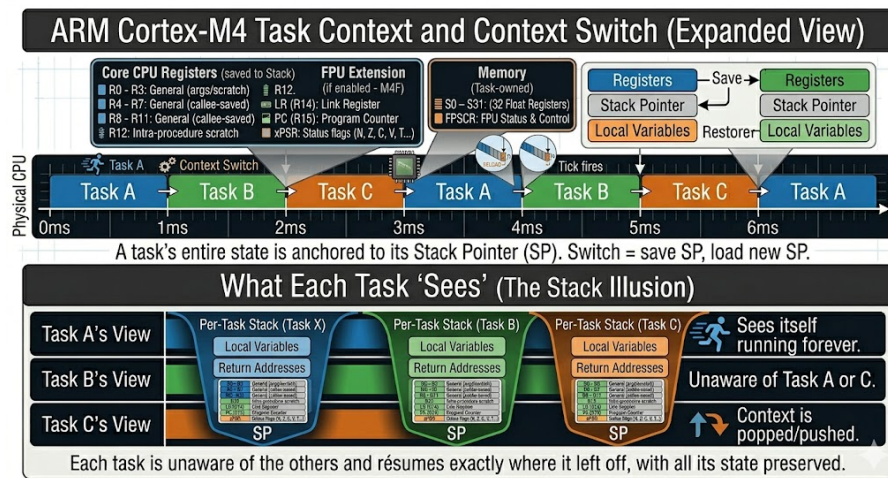


Figure 3.2: Task Context on ARM Cortex-M4

3.3 The Task Control Block (TCB)

The TCB is a data structure the RTOS maintains for every task. Think of it as a task’s “file” in the OS. At minimum it must store the saved stack pointer so the context switch knows where to find the saved registers.

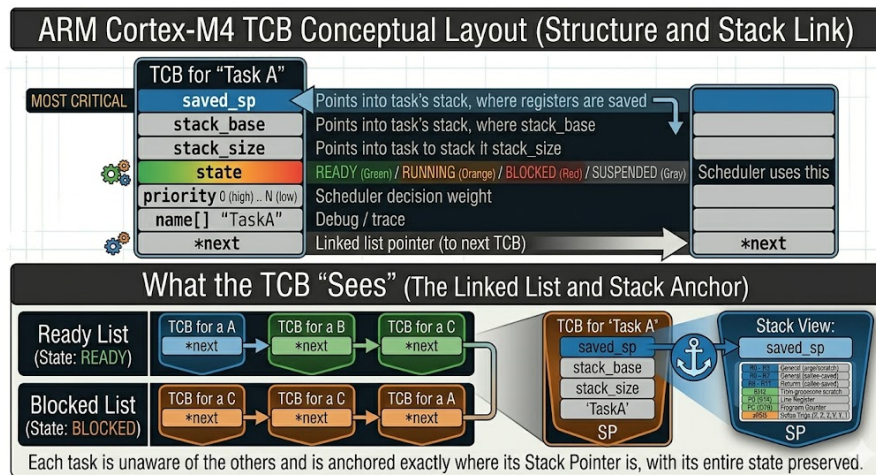


Figure 3.3: Task Control Block — Conceptual Layout

In FreeRTOS this struct is called TCB_t (defined in tasks.c). The first field is always pxTopOfStack — because the context switch assembly code must find it at offset 0 without needing to know the full struct layout.

3.3.1 Code: Minimal TCB definition

```
1 /*
2  * tcb.h - Minimal Task Control Block for a home-grown RTOS demo
3  *
4  * This is a teaching implementation. FreeRTOS's TCB_t is larger and
```

```

5  * richer, but every field here maps to something in the real thing.
6  */
7
8  #ifndef TCB_H
9  #define TCB_H
10
11 #include <stdint.h>
12 #include <stddef.h>
13
14 /* Task states */
15 typedef enum {
16     TASK_READY      = 0, /* Ready to run, waiting for CPU */
17     TASK_RUNNING    = 1, /* Currently executing on CPU */
18     TASK_BLOCKED    = 2, /* Waiting for an event (semaphore, time) */
19     TASK_SUSPENDED  = 3, /* Explicitly suspended, not schedulable */
20 } TaskState_t;
21
22 /* Task function signature: takes a void* arg, never returns */
23 typedef void (*TaskFunc_t)(void *arg);
24
25 /* Task Control Block */
26 typedef struct TCB_s {
27     /*
28      * MUST be first field – context switch assembly addresses
29      * this at offset 0 without knowing the struct layout.
30      */
31     volatile uint32_t *saved_sp; /* Saved stack pointer */
32
33     /* Stack boundaries */
34     uint32_t *stack_base; /* Lowest valid stack address */
35     uint32_t stack_size; /* Stack size in bytes */
36
37     /* Scheduler fields */
38     TaskState_t state; /* Current state */
39     uint8_t priority; /* 0 = highest, 255 = lowest */
40     uint32_t wake_tick; /* Tick to unblock at (for delay) */
41
42     /* Identity */
43     const char *name; /* Human-readable task name */
44
45     /* Linked list for scheduler queues */
46     struct TCB_s *next; /* Next TCB in ready/blocked list */
47 } TCB_t;
48
49 #endif /* TCB_H */

```

3.4 Task States and Transitions

Every task is always in one of four states. The scheduler only ever runs tasks in the **READY** state.

RTOS TASK STATE TRANSITION DIAGRAM

Conceptual Model of Task Lifecycle within a Scheduler

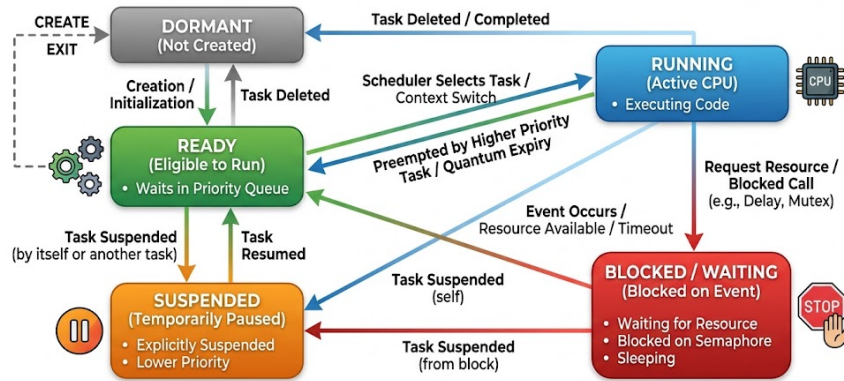


Figure 3.4: Task State Machine



- Only ONE task is RUNNING at a time (on a single core).
- Multiple tasks can be READY simultaneously — priority breaks ties.
- A BLOCKED task consumes zero CPU time.
- SUSPENDED tasks do not wake on events — only on explicit Resume().

3.5 The Task Stack

Each task gets its **own private stack** — a region of SRAM reserved exclusively for that task's local variables, function call chain, and saved register frame.

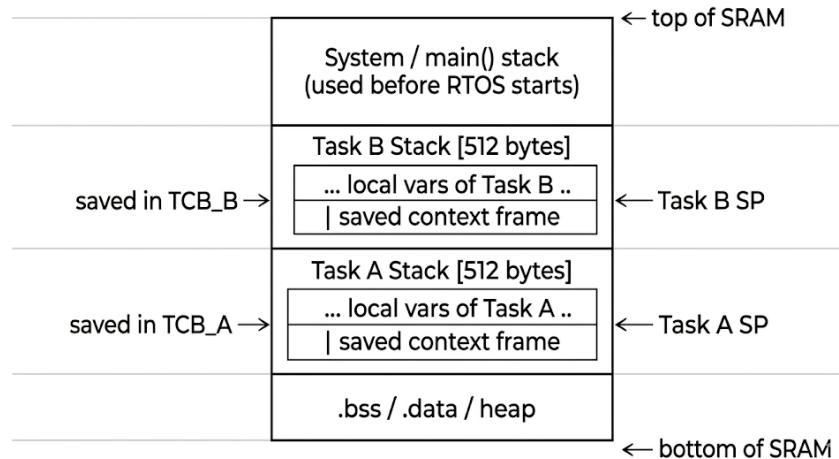


Figure 3.5: Task stack



Critical property: - Task A cannot corrupt Task B's stack. - Task B cannot corrupt Task A's local variables.

3.5.1 Code: Initialising a task stack

When a task is created, its stack must be pre-initialised to look exactly like it was already interrupted mid-execution. The context switch will then “restore” this fake frame and the task will start running.

```
1  /*
2  * task_stack_init.c
3  *
4  * Pre-fill a task's stack so that when the context switch restores
5  * it for the first time, the CPU jumps to task_func(arg).
6  *
7  * ARM Cortex-M stack frame layout (grows downward):
8  *
9  * [Higher address] (stack top on entry)
10 * +-----+
11 * | xPSR   | Must have bit 24 set (Thumb mode)
12 * | PC     | < task_func - where task starts
13 * | LR     | < task_exit_handler (called if task returns)
14 * | R12    | 0x00000000 (dummy)
15 * | R3     | 0x00000000
16 * | R2     | 0x00000000
17 * | R1     | 0x00000000
18 * | R0     | < arg (task function argument)
19 * +-----+ < hardware-pushed frame ends here
20 * | R11    | \
21 * | R10    | |
22 * | R9     | | Software-pushed (callee-saved registers)
23 * | R8     | | pushed by our context switch code
24 * | R7     | |
25 * | R6     | |
26 * | R5     | |
27 * | R4     | /
28 * +-----+ < saved_sp points here after init
29 * [Lower address]
30 */
31
32 #include <stdint.h>
33 #include "tcb.h"
34
35 /* Called if a task function ever returns - should never happen */
36 static void task_exit_handler(void)
37 {
38     /* In production: delete the current task.
39      * Here: trap in debugger. */
40     while (1);
41 }
42
43 /*
44 * stack_init() - Initialise a task stack with a fake exception frame.
45 *
46 * Parameters:
47 *   stack_top : pointer to ONE PAST the top of the stack buffer
48 *               (stacks grow downward, so top = buffer_base + size)
49 *   task_func : the task function to run
50 *   arg       : passed as R0 (first argument) to task_func
51 *
52 * Returns: the initial stack pointer to store in TCB.saved_sp
53 */
54 uint32_t *stack_init(uint32_t *stack_top,
```

```

55         TaskFunc_t   task_func,
56         void         *arg)
57 {
58     /*
59      * Cortex-M requires 8-byte stack alignment at exception entry.
60      * Ensure stack_top is 8-byte aligned before we start filling.
61      */
62     uint32_t *sp = (uint32_t *)((uintptr_t)stack_top & ~0x7UL);
63
64     /* -- Hardware-pushed exception frame (filled in reverse) -- */
65
66     *--sp = 0x01000000UL;          /* xPSR: Thumb bit (bit 24) set */
67     *--sp = (uint32_t)task_func;   /* PC:   task entry point       */
68     *--sp = (uint32_t)task_exit_handler; /* LR: if task returns   */
69     *--sp = 0x00000000UL;          /* R12: scratch           */
70     *--sp = 0x00000000UL;          /* R3                      */
71     *--sp = 0x00000000UL;          /* R2                      */
72     *--sp = 0x00000000UL;          /* R1                      */
73     *--sp = (uint32_t)arg;          /* R0: task argument       */
74
75     /* -- Software-saved callee registers (our context switch saves these) -- */
76     *--sp = 0x00000000UL;          /* R11 */
77     *--sp = 0x00000000UL;          /* R10 */
78     *--sp = 0x00000000UL;          /* R9  */
79     *--sp = 0x00000000UL;          /* R8  */
80     *--sp = 0x00000000UL;          /* R7  */
81     *--sp = 0x00000000UL;          /* R6  */
82     *--sp = 0x00000000UL;          /* R5  */
83     *--sp = 0x00000000UL;          /* R4  */
84
85     /* sp now points to the top of the initialised frame.
86      * Store this in TCB.saved_sp. The context switch will
87      * pop R4-R11 then do an exception return to pop the rest. */
88     return sp;
89 }

```

3.6 The Context Switch — Heart of the RTOS

The context switch is the code that swaps from one task to another. It must:

1. Save the **current** task's CPU registers onto its stack, update its `saved_sp`.
2. Pick the **next** task to run (the scheduler's job).
3. Load the **next** task's `saved_sp`, pop its saved registers.
4. Return — which resumes the next task mid-execution.

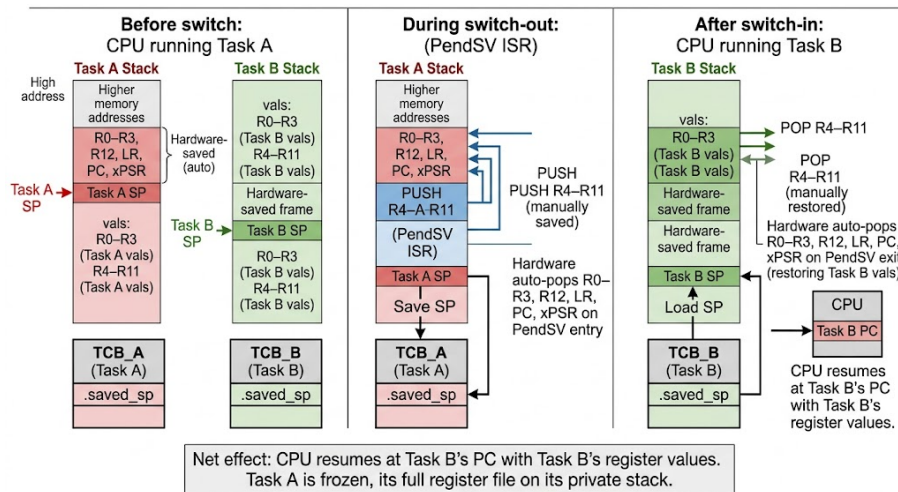


Figure 3.6: Context Switch — Register Save and Restore

On ARM Cortex-M, the context switch is triggered by the **PendSV** exception (Pended Supervisor Call). FreeRTOS and most other RTOSes use PendSV at the lowest possible priority so that all real ISRs complete before the switch occurs.

3.6.1 Code: Context switch in assembly (ARM Cortex-M, GCC syntax)

```

1  /*
2  * context_switch.s - Minimal PendSV context switch for ARM Cortex-M
3  *
4  * Globals used:
5  *   current_tcb - pointer to TCB_t* of the currently running task
6  *   next_tcb   - pointer to TCB_t* of the task to switch to
7  *               (set by the scheduler before pending PendSV)
8  *
9  * TCB layout assumption:
10 *   offset 0 = saved_sp (uint32_t *)
11 */
12
13 .syntax unified
14 .thumb
15
16 .extern current_tcb /* TCB_t **current_tcb */
17 .extern next_tcb   /* TCB_t **next_tcb   */
18
19 .global PendSV_Handler
20 .type PendSV_Handler, %function
21
22 PendSV_Handler:
23 /*
24  * On PendSV entry, hardware has already pushed:
25  *   xPSR, PC, LR, R12, R3, R2, R1, R0
26  *   onto the current task's stack (MSP or PSP).
27  *
28  * We use PSP (Process Stack Pointer) for tasks and
29  * MSP (Main Stack Pointer) for the RTOS kernel.
30  * This is the standard Cortex-M RTOS model.
31  */
32
33 /* 1. Disable interrupts during the switch */

```

```

34 CPSID    I
35
36 /* 2. Get the current task's PSP (process stack pointer) */
37 MRS      R0, PSP
38
39 /* 3. Save callee-saved registers R4-R11 onto PSP stack
40  *   STMDB = Store Multiple, Decrement Before (push descending) */
41 STMDB    R0!, {R4-R11}
42
43 /* 4. Save updated SP into current task's TCB
44  *   current_tcb is TCB_t** - load the pointer, then store SP */
45 LDR      R1, =current_tcb
46 LDR      R1, [R1]          /* R1 = *current_tcb (= TCB_t *) */
47 STR      R0, [R1, #0]      /* TCB.saved_sp = R0 (updated PSP) */
48
49 /* 5. Load next task's SP from its TCB
50  *   next_tcb was set by the scheduler before PendSV was pended */
51 LDR      R2, =next_tcb
52 LDR      R2, [R2]          /* R2 = *next_tcb (= TCB_t *) */
53 LDR      R0, [R2, #0]      /* R0 = next TCB.saved_sp */
54
55 /* 6. Update current_tcb = next_tcb */
56 LDR      R3, =current_tcb
57 STR      R2, [R3]
58
59 /* 7. Restore callee-saved registers R4-R11 from next task's stack
60  *   LDMIA = Load Multiple, Increment After (pop ascending) */
61 LDMIA    R0!, {R4-R11}
62
63 /* 8. Write the new SP back to PSP */
64 MSR      PSP, R0
65
66 /* 9. Re-enable interrupts */
67 CPSIE    I
68
69 /*
70  * 10. Return from exception using EXC_RETURN.
71  *   LR still holds EXC_RETURN (set by hardware on entry).
72  *   BX LR triggers hardware unstacking of
73  *   R0-R3, R12, LR, PC, xPSR from the NEW task's stack.
74  *   CPU jumps to next task's PC - the switch is complete.
75  */
76 BX      LR
77
78 .size PendSV_Handler, .-PendSV_Handler

```

3.7 Task Creation — Wiring It All Together

With the TCB, stack initialisation, and context switch defined, task creation becomes straightforward:

```

1 /*
2  * task_create.c - Allocate a TCB, initialise its stack, add to ready list
3  */
4
5 #include <string.h>
6 #include "tcb.h"
7

```

```

8  /* Statically allocated TCB pool (no heap required) */
9  #define MAX_TASKS 8
10 static TCB_t tcb_pool[MAX_TASKS];
11 static uint8_t tcb_count = 0;
12
13 /* Head of the ready list – singly linked, priority sorted */
14 TCB_t *ready_list_head = NULL;
15
16 /* Scheduler's current task pointer */
17 TCB_t *current_tcb = NULL;
18 TCB_t *next_tcb = NULL;
19
20 /* Declare stack_init from task_stack_init.c */
21 extern uint32_t *stack_init(uint32_t *, TaskFunc_t, void *);
22
23 /*
24  * task_create() – Create a new task.
25  *
26  * Parameters:
27  *   func      : task function void func(void *arg)
28  *   arg       : passed to func as first argument
29  *   stack_buf  : caller-provided stack buffer
30  *   stack_size : size of stack_buf in bytes
31  *   priority   : scheduling priority (0 = highest)
32  *   name      : human-readable name (debug only)
33  *
34  * Returns pointer to TCB, or NULL if the pool is exhausted.
35  */
36 TCB_t *task_create(TaskFunc_t func,
37                   void *arg,
38                   uint32_t *stack_buf,
39                   uint32_t stack_size,
40                   uint8_t priority,
41                   const char *name)
42 {
43     if (tcb_count >= MAX_TASKS) return NULL;
44
45     TCB_t *tcb = &tcb_pool[tcb_count++];
46     memset(tcb, 0, sizeof(*tcb));
47
48     tcb->stack_base = stack_buf;
49     tcb->stack_size = stack_size;
50     tcb->priority = priority;
51     tcb->name = name;
52     tcb->state = TASK_READY;
53
54     /* Calculate stack top and pre-fill the exception frame */
55     uint32_t *stack_top = stack_buf + (stack_size / sizeof(uint32_t));
56     tcb->saved_sp = stack_init(stack_top, func, arg);
57
58     /* Insert into ready list, sorted by priority (lower number first) */
59     TCB_t **pp = &ready_list_head;
60     while (*pp && (*pp)->priority <= priority) {
61         pp = &(*pp)->next;
62     }
63     tcb->next = *pp;
64     *pp = tcb;
65

```

```

66     return tcb;
67 }
68
69 /* -- Usage example -- */
70
71 /* Stack buffers – statically declared, sized per task's needs */
72 static uint32_t stack_led[256];    /* 256 words = 1024 bytes */
73 static uint32_t stack_uart[512];   /* 512 words = 2048 bytes */
74 static uint32_t stack_sensor[256];
75
76 /* Task functions – each written as a simple infinite loop */
77 void task_led(void *arg)
78 {
79     (void)arg;
80     while (1)
81     {
82         led_toggle();
83         vTaskDelay(500); /* yield CPU for 500 ticks – covered in Ch 5 */
84     }
85 }
86
87 void task_uart(void *arg)
88 {
89     (void)arg;
90     uint8_t byte;
91     while (1)
92     {
93         if (uart1_read_byte(&byte))
94             uart1_send_byte(byte);
95         vTaskDelay(1);
96     }
97 }
98
99 void task_sensor(void *arg)
100 {
101     (void)arg;
102     while (1)
103     {
104         uint16_t adc_val = adc_read();
105         process_sensor(adc_val);
106         vTaskDelay(10);
107     }
108 }
109
110 void rtos_main(void)
111 {
112     /* Create tasks – each with its own private stack */
113     task_create(task_led,    NULL, stack_led,    sizeof(stack_led),    2, "LED");
114     task_create(task_uart,   NULL, stack_uart,   sizeof(stack_uart),   1, "UART");
115     task_create(task_sensor, NULL, stack_sensor, sizeof(stack_sensor), 0, "Sensor");
116
117     /* Start the scheduler – never returns (Chapter 5) */
118     scheduler_start();
119 }

```

Notice how each task is now **sequential, readable code**. No state machines. No `if (elapsed >= 500)` checks. Each task just does its job and calls `vTaskDelay()` to yield. The RTOS handles the rest.

3.8 The Ready List — How the Scheduler Sees Tasks

The scheduler maintains lists of TCBs. The simplest structure is a priority-sorted linked list of all READY tasks.

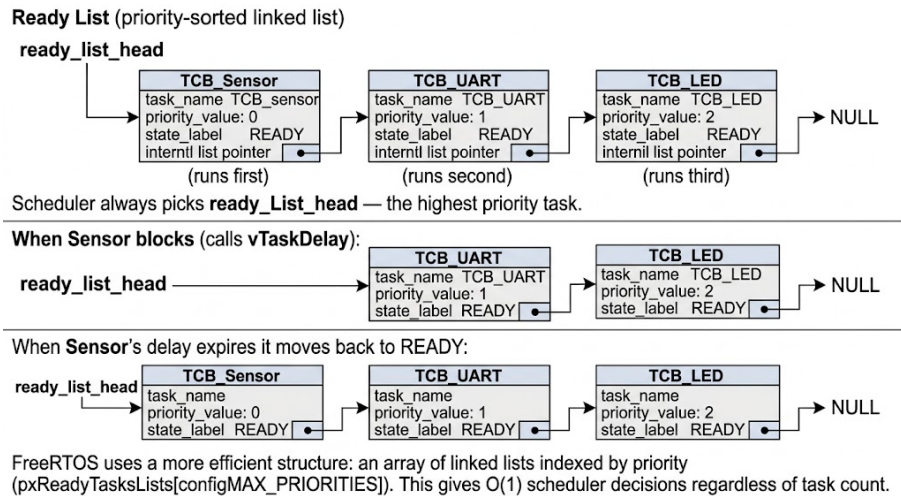


Figure 3.7: Ready List

3.9 The Unsolved Tension

We now have tasks, each with its own stack, its own context, and written as simple sequential code. The illusion of parallelism is working. But a new question has emerged:

The Problem of "Who Runs Next?"

Three tasks, all READY:

Sensor P=0	UART P=1	LED P=2
READY	READY	READY

Questions that remain unanswered:

1. Does Sensor run forever, starving UART and LED?
2. How long does each task get before it is switched out?
3. What if Sensor has nothing to do – does it still hold the CPU?
4. Who triggers the switch – the task itself or the RTOS?
5. What happens when two tasks have the same priority?

These questions define SCHEDULING POLICY – the subject of Chapter 4.

End of Chapter 3. We can create tasks, switch between them, and each task believes it runs alone. The missing piece is the policy that governs which task runs and for how long. That is the scheduler — and it is what Chapter 4 builds.



Chapter 4 Preview — The Scheduler We will implement preemptive and cooperative scheduling from scratch, understand the tick interrupt as a scheduling heartbeat, explore priority-based and round-robin policies, and see exactly what FreeRTOS's scheduler does inside `vTaskSwitchContext()`.

Chapter 4

Concepts and Recommendations

4.1 The Pitfalls of Dynamic Memory (malloc)

In a standard PC application, you use malloc and free without much thought. In an RTOS, this is often forbidden or highly discouraged.

- **Non-Determinism:** malloc takes a variable amount of time to find a free block. In a real-time system, you need to know exactly how long an operation takes.
- **Fragmentation:** Over time, allocating and freeing small chunks of memory creates “holes.” Eventually, you might have 10KB of free memory total, but not a single continuous 2KB block, causing the system to crash.
- **Solution:** Most RTOS developers use Static Allocation or Memory Pools (fixed-size blocks) to ensure memory is always available and allocation time is constant.

4.2 Task States and Timing Analysis

Understanding the State Transition Diagram is crucial. A task isn’t just “running” or “not running.”

- **Ready:** The task is prepared to run but a higher-priority task is currently using the CPU.
- **Running:** The task is currently executing instructions.
- **Blocked/Waiting:** The task is waiting for an event (a timer, a semaphore, or a queue).
- **Suspended:** The task is manually paused and won’t run until another task “resumes” it.

The Interview Question: “What happens if a high-priority task never enters the Blocked state?”

The Answer: It “starves” the system. Lower-priority tasks (including the Idle task which often handles power saving) will never get a single CPU cycle.

4.3 Queues: The Right Way to Pass Data

While Semaphores are great for signals, Queues are the gold standard for data.

- **Copy by Value vs. Copy by Reference:** Small data (like a sensor integer) is copied directly into the queue. For large data (like a 1MB image buffer), you put the pointer to the data in the queue.
- **Thread Safety:** Queues are internally protected by the RTOS. You don’t need to wrap a queue in a Mutex; the RTOS handles the locking for you.

4.4 Software Timers vs. Hardware Timers

- **Hardware Timers:** Use high-precision peripherals to trigger ISRs. Best for microsecond-level accuracy (e.g., PWM or high-speed sampling).
- **Software Timers:** Managed by the RTOS “Timer Task.” They are easier to use and don’t consume hardware resources, but their accuracy is limited by the RTOS Tick Rate (usually 1ms). If your tick is 1ms, you cannot have a software timer trigger every 0.5ms.

4.5 The Tick Hook & System Heartbeat

Every RTOS relies on a hardware timer (usually called the SysTick in ARM Cortex-M) to create a periodic interrupt.

- **The Problem:** You need to perform a small action (like blinking a status LED or updating a watchdog timer) every 1ms, but you don’t want to create a whole new task for it.
- **The Concept:** The Tick Hook is a callback function inside the RTOS kernel that executes every single time the system tick interrupt occurs.
- **The Challenge:** Since this runs inside an ISR context, you must be extremely careful. If the Tick Hook takes too long, the whole system drifts in time.

4.6 The Idle Task & Power Management

When no tasks are in the “Ready” state (e.g., everyone is blocked waiting for a sensor or a timer), the RTOS doesn’t just stop. It runs the Idle Task.

- **The Problem:** A battery-powered device needs to last for months. If the CPU is always “Running” (even if just in a loop), the battery will die in days.
- **The Concept:** Tickless Idle. Instead of waking up every 1ms to check the scheduler, the RTOS calculates how long it can safely “sleep” until the next scheduled event and puts the CPU into a Deep Sleep mode.
- **The Interview Question:** “What is the priority of the Idle Task, and why?”
- **The Answer:** It is always Priority 0 (the lowest). If it were higher, it would block every other task from running.

4.7 Event Groups (The “Wait for All” logic)

Sometimes, a task needs to wait for multiple things to happen before it can proceed. For example: “Start the Motor only if (Button is Pressed) AND (Temperature is OK) AND (Safety Cover is Closed).”

- **The Problem:** Using three separate semaphores is inefficient and can lead to complex code.
- **The Concept:** Event Groups / Event Flags.

Chapter 5

Problem solving

5.1 Odd / Even Print

- **Problem:** Create a two tasks where one task print odd number and another prints even number but while printing the numbers should be in 1, 2, 3, 4, ... and So on.

Code

```
1 SemaphoreHandle_t semOdd = 1; // Start with 1 so Odd Task runs first
2 SemaphoreHandle_t semEven = 0; // Start with 0 so Even Task blocks
3
4 void Task_Odd(void) {
5     int count = 1;
6     while(1) {
7         take(semOdd);           // Wait for turn
8         print(count);
9         count += 2;
10        give(semEven);          // Signal Even task
11    }
12 }
13
14 void Task_Even(void) {
15     int count = 2;
16     while(1) {
17         take(semEven);          // Wait for turn
18         print(count);
19         count += 2;
20         give(semOdd);           // Signal Odd task
21    }
22 }
```

5.2 The Producer-Consumer Pattern

- **The Problem:** Task A reads data from a sensor at a high frequency and places it in a shared circular buffer. Task B processes that data and sends it over UART.
- **The Challenge:** How do you ensure Task B doesn't read an empty buffer and Task A doesn't overwrite data that Task B hasn't processed yet? Which RTOS primitive (Binary Semaphore, Counting Semaphore, or Mutex) is best for signaling Task B that data is ready?
- **Concept:** Use counting semaphores to track empty/full slots and a mutex for data integrity.

Code:


```

1 Semaphore_t semEmpty = BUFFER_SIZE; // Tracks available space
2 Semaphore_t semFull = 0; // Tracks available data
3 Mutex_t bufferMutex; // Protects the buffer indices
4
5 void Producer(void) {
6     while(1) {
7         data = read_sensor();
8
9         take(semEmpty); // Wait if buffer is full
10        lock(bufferMutex);
11
12        buffer[in] = data;
13        in = (in + 1) % SIZE;
14
15        unlock(bufferMutex);
16        give(semFull); // Signal Consumer that data is ready
17    }
18 }
19
20 void Consumer(void) {
21     while(1) {
22         take(semFull); // Wait if buffer is empty
23         lock(bufferMutex);
24
25         data = buffer[out];
26         out = (out + 1) % SIZE;
27
28         unlock(bufferMutex);
29         give(semEmpty); // Signal Producer that space is free
30
31         process(data);
32     }
33 }

```

5.3 Queue Overflow

- **The Problem:** A High-Priority Sensor Task is producing data faster than a Low-Priority Display Task can process and render it. Because the queue has a fixed size, it eventually hits its limit (becomes full).
- **The Challenge:** If the High-Priority task tries to send data to a full queue, it faces a dilemma:
 - **Block:** If it waits for space, the high-priority task stops running, potentially missing critical sensor samples (Timing Jitter).
 - **Discard:** If it skips the data, you lose information.
 - **Overwrite:** If it forces the data in, it might corrupt the message sequence.

The challenge is maintaining system responsiveness without losing the “freshest” data.

- **Concept:** Ping Pong Buffer or Double buffering

Code

```

1 #define BUF_SIZE 100
2
3 // Two physical memory areas
4 float Buffer0[BUF_SIZE];
5 float Buffer1[BUF_SIZE];
6
7 // Signals to tell the Display task which buffer is ready

```

```

8 SemaphoreHandle_t semBuffer0Ready;
9 SemaphoreHandle_t semBuffer1Ready;
10 // Signal to tell Sensor task a buffer is empty and safe to use
11 SemaphoreHandle_t semBufferProcessed;
12
13 void Sensor_Task(void) {
14     float *currentBuffer = Buffer0;
15     int index = 0;
16
17     while(1) {
18         currentBuffer[index++] = Read_Sensor();
19
20         if (index >= BUF_SIZE) {
21             // Buffer is full!
22             if (currentBuffer == Buffer0) {
23                 give(semBuffer0Ready); // Tell Display to process Buffer 0
24                 currentBuffer = Buffer1; // Immediately switch to Buffer 1
25             } else {
26                 give(semBuffer1Ready); // Tell Display to process Buffer 1
27                 currentBuffer = Buffer0; // Immediately switch to Buffer 0
28             }
29             index = 0;
30
31             // Wait until the Display task has finished at least ONE of the buffers
32             // This provides the 'Zero Data Loss' protection
33             take(semBufferProcessed);
34         }
35         TaskDelayMs(1);
36     }
37 }
38
39 void Display_Task(void) {
40     while(1) {
41         // Check if Buffer 0 is ready
42         if (take(semBuffer0Ready, 0) == SUCCESS) {
43             Render_Graph(Buffer0, BUF_SIZE);
44             give(semBufferProcessed); // Signal that Buffer 0 is now free
45         }
46         // Check if Buffer 1 is ready
47         else if (take(semBuffer1Ready, 0) == SUCCESS) {
48             Render_Graph(Buffer1, BUF_SIZE);
49             give(semBufferProcessed); // Signal that Buffer 1 is now free
50         }
51     }
52 }

```

5.4 Handling Shared Hardware (The Re-entrancy Problem)

- **The Problem:** Two different tasks (High Priority and Low Priority) both need to write log messages to the same SPI Flash memory.
- **The Challenge:**
 - If the Low Priority task is mid-write and the High Priority task preempts it and tries to write to SPI, the hardware state will crash. How do you protect this shared resource?
 - Why would a Mutex be preferred over a simple Global Boolean flag?
 - * **Atomicity:** Checking a flag and then setting it is a “Read-Modify-Write” operation. In assembly,

this is multiple instructions. A task could check the flag, see it's false, and get preempted before it sets it to true. Another task could then run, see the same false flag, and you now have two tasks accessing the same hardware simultaneously. A Mutex uses hardware-level atomic instructions (like Test-and-Set) to ensure this cannot happen.

- **Blocking vs. Polling:** With a boolean flag, a task has to “spin” (loop constantly) to check the flag, wasting CPU cycles. A Mutex puts the task into a Blocked state, allowing other tasks to run. The RTOS scheduler only wakes the task up when the Mutex becomes available.
- **Concept:** Protecting a non-reentrant peripheral using a Mutex. Mutex is preferred over a simple global boolean because priority inversion is handled by mutex using priority inheritance.

Code:

```
1  Mutex_t spiMutex;
2
3  void Log_Data(char* msg) {
4      lock(spiMutex);          // Only one task can own the SPI bus
5
6      SPI_Select_Chip();
7      SPI_Write(msg);
8      SPI_Deselect_Chip();
9
10     unlock(spiMutex);        // Release for other tasks
11 }
12
13 void Task_HighPriority(void) {
14     Log_Data("High Priority Alert!");
15 }
16
17 void Task_LowPriority(void) {
18     Log_Data("Routine system log.");
19 }
```

5.5 The Deadlock Scenario

- **The Problem:** Task 1 needs Resource A then Resource B. Task 2 needs Resource B then Resource A.
- **The Challenge:** Describe the “Deadly Embrace” where both tasks end up waiting forever. What are two architectural ways to prevent this in a real-time system?
 - **Resource Hierarchy (Lock Ordering):** Assign a numerical “rank” to every resource (e.g., SPI=1, I2C=2, UART=3). Rule: A task can only acquire a resource with a higher rank than the ones it already holds. This makes it mathematically impossible to form a circular wait.
 - **Timeout-Based Acquisition:** Instead of waiting forever (wait_forever), use a timeout (e.g., take_mutex(lock, 100ms)). If the task can't get all the resources it needs within the time limit, it must release all its held locks and try again later. This breaks the “Hold and Wait” condition of deadlocks.
- **Concept:** To prevent circular wait, tasks must acquire resources in the exact same numerical order.

Code:

```
1  Mutex_t Mutex_A; // Resource 1
2  Mutex_t Mutex_B; // Resource 2
3
4  void Task_Safe(void) {
5      while(1) {
6          // ALWAYS take A before B
7          lock(Mutex_A);
8          lock(Mutex_B);
```

```

9
10     // Access shared resources
11
12     unlock(Mutex_B);
13     unlock(Mutex_A);
14 }
15 }

```

5.6 Priority Inversion:

- **The Problem:** A High-priority task is waiting for a Mutex held by a Low-priority task. A Medium-priority task starts running, preventing the Low-priority task from finishing and releasing the Mutex.
- **The Challenge:** How does the “Priority Inheritance” protocol solve this? Can you explain the difference between Priority Inheritance and Priority Ceiling?
 - **Priority Inheritance:** The priority of the lower-priority task holding the lock is boosted only when a higher-priority task tries to acquire that same lock. It is “reactive.”
 - **Priority Ceiling:** Every resource (Mutex) is assigned a “Ceiling” priority (the priority of the highest task that could ever use it). As soon as any task grabs that Mutex, its priority is immediately boosted to that Ceiling, regardless of whether a higher-priority task is actually waiting. This is “proactive” and effectively prevents deadlocks as well.
- **Concept:** Illustrating how a Mutex with inheritance automatically fixes the “Unbounded” inversion.

Code

```

1  /*
2   RTOS Settings: Mutex must be created with Priority Inheritance ENABLED.
3   Low Priority (LP) holds Mutex.
4   High Priority (HP) wants Mutex.
5   LP is boosted to HP's priority until it unlocks.
6  */
7
8  void Task_Low(void) {
9      lock(mutexInherit);
10     // Even if Task_Medium tries to preempt here,
11     // it fails because Task_Low is now "High Priority"
12     unlock(mutexInherit); // Priority returns to Low
13 }

```

5.7 Interrupt to Task Communication

- **The Problem:** An external GPIO interrupt (ISR) triggers every time a button is pressed. You want to toggle an LED in a Task.
- **The Challenge:** Why shouldn’t you toggle the LED or perform heavy logic directly inside the ISR? Explain the concept of “Deferred Interrupt Processing” using a semaphore or task notification.
 - Why avoid heavy logic/LED toggling in ISRs?
 1. **Jitter:** While an ISR is running, other interrupts of the same or lower priority are blocked. This creates “latency” for other parts of the system.
 2. **Stack Overflow:** ISRs often use a limited system stack. Complex logic or printf calls can easily crash the system.
 3. **Non-deterministic behavior:** Many RTOS functions (like delay or lock_mutex) can block. You cannot block inside an ISR, as there is no “task context” to switch away from.

- **Deferred Interrupt Processing:** Instead of doing the work in the ISR, the ISR simply “gives” a semaphore or a task notification and exits. The RTOS then sees a high-priority “Worker Task” is now ready to run. The worker task wakes up, does the heavy lifting (like writing to an SD card or toggling an LED), and then goes back to sleep. This ensures the hardware interrupt is cleared in microseconds, keeping the system responsive.

Concept: Keep the ISR short by offloading logic to a task using a binary semaphore.

Code

```
1 Semaphore_t semButton;
2
3 // ISR (Interrupt Service Routine)
4 void GPIO_ISR(void) {
5     // Keep this extremely fast!
6     give_from_ISR(semButton); // Wake up the task
7 }
8
9 // Processing Task
10 void Task_ButtonHandler(void) {
11     while(1) {
12         take(semButton);    // Blocks here until button is pressed
13         // Perform heavy work (e.g., Debouncing, Network requests)
14         Toggle_LED();
15         printf("Button Pressed!\n");
16     }
17 }
```

Chapter 6

Zephyr RTOS in Practice: Build Scalable Embedded Application

6.1 Introduction

6.1.1 What is Zephyr?

- A scalable, open-source RTOS for embedded systems.
- Supports microcontrollers, IoT devices, wearables, and edge systems.
- Backed by a large industry consortium through the Linux Foundation.
- Actively used in real-world commercial products.



A modern, industry-backed RTOS for real-world IoT products.

6.1.2 Where Zephyr Comes From?

- Born from the merger of two RTOS projects: Rocket RTOS and Wind River's Microkernel.
- Adopted by the Linux Foundation in 2016.
- Designed as a modern alternative to legacy RTOS systems with stronger security, modularity, and tooling.

Year	Event
2001	Wind River microkernel introduced
2015	Rocket RTOS open-sourced
2016	Zephyr adopted by Linux Foundation
2019	Major modularity improvements
2021+	Rapid ecosystem growth



A modern, unified RTOS built to replace fragmented legacy systems.

6.1.3 Zephyr's Key Characteristics

- Highly modular kernel (choose only what your application needs).
- Memory-safe design with strong security primitives.
- Unified device driver model with devicetree support.
- Multi-architecture: ARM, RISC-V, Xtensa, x86, ARC, MIPS.

- First-class networking: BLE, Wi-Fi, Ethernet, Thread, Matter.



A scalable, secure RTOS platform with unified APIs and rich networking.

6.1.4 Why You Should Learn Zephyr

- Industry demand for engineers who understand modern RTOS design.
- Works across many chips and vendors, so your skills transfer easily.
- Excellent documentation, CI-tested code, and long-term stability.
- Powerful abstraction layers reduce vendor lock-in.
- Ideal for commercial IoT products that require security + maintainability.



Zephyr gives you portable, in-demand skills for modern IoT products.

6.1.5 Zephyr from the Arduino Perspective

- Arduino is easy to use and ideal for beginners, but not designed for complex RTOS tasks.
- Arduino “sketches” run a single, long loop with limited concurrency.
- Peripheral handling is simplified, but lacks structured device drivers found in Zephyr.
- Memory management is minimal, making larger systems harder to scale safely.
- No built-in RTOS features like threads, scheduling, IPC, or power-aware execution.
- Zephyr offers a professional-grade environment while still supporting many Arduino-class MCUs.



Arduino is great for quick prototyping; Zephyr offers structure.

6.1.6 Why Arduino Users Should Learn Zephyr

- Move from single-loop programs to real multitasking.
- Learn industry-standard device driver models.
- Gain experience with networking stacks like Wi-Fi, BLE, Thread, and Matter.
- Understand how commercial IoT firmware is structured.



Zephyr upgrades your skills to industry-level embedded engineering.

6.1.7 Zephyr Architecture Overview

- Modular RTOS built around a small, efficient microkernel.
- Hardware abstraction through drivers and devicetree.
- Subsystems for networking, sensors, storage, Bluetooth, power management, and more.
- Clear separation between kernel, drivers, and user application code.

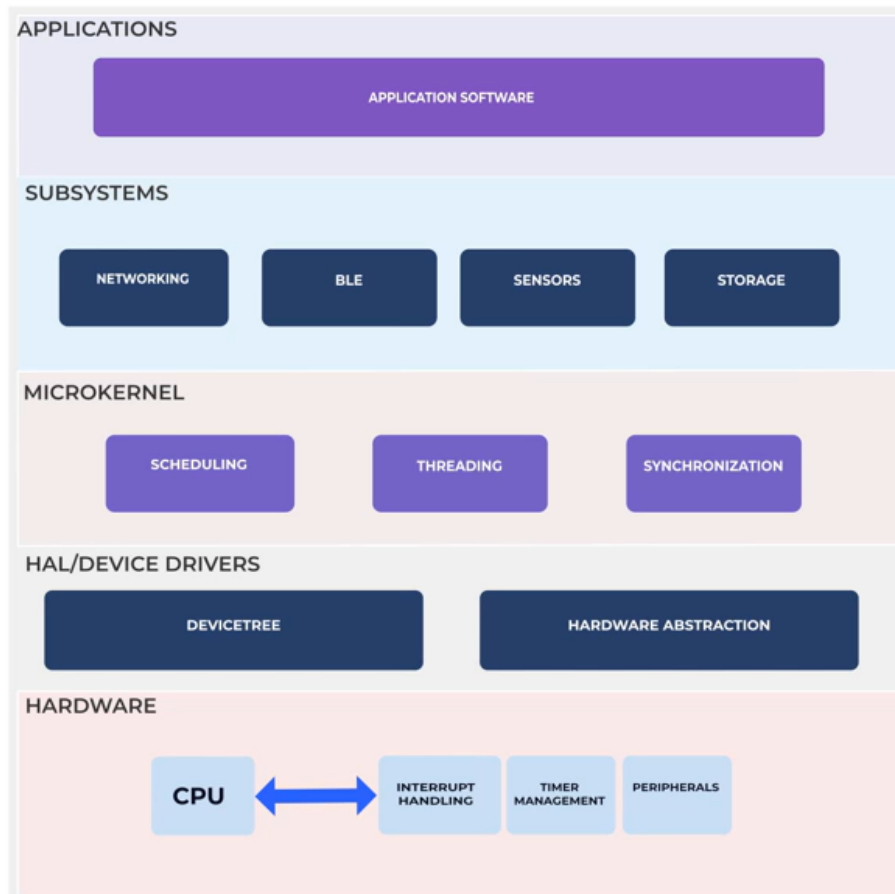


Figure 6.1: Zephyr Architecture Overview



Zephyr layers a small kernel with selectable modular services.

6.1.8 How Zephyr Projects Are Structured

- Standard layout: `src/` , `prj.conf` , `CMakeLists.txt` , and optional Devicetree overlays.
- `prj.conf` controls compile-time configuration through Kconfig.
- Devicetree overlays describe hardware, buses, and device connections.
- Build system uses CMake and `west` to generate the final executable.

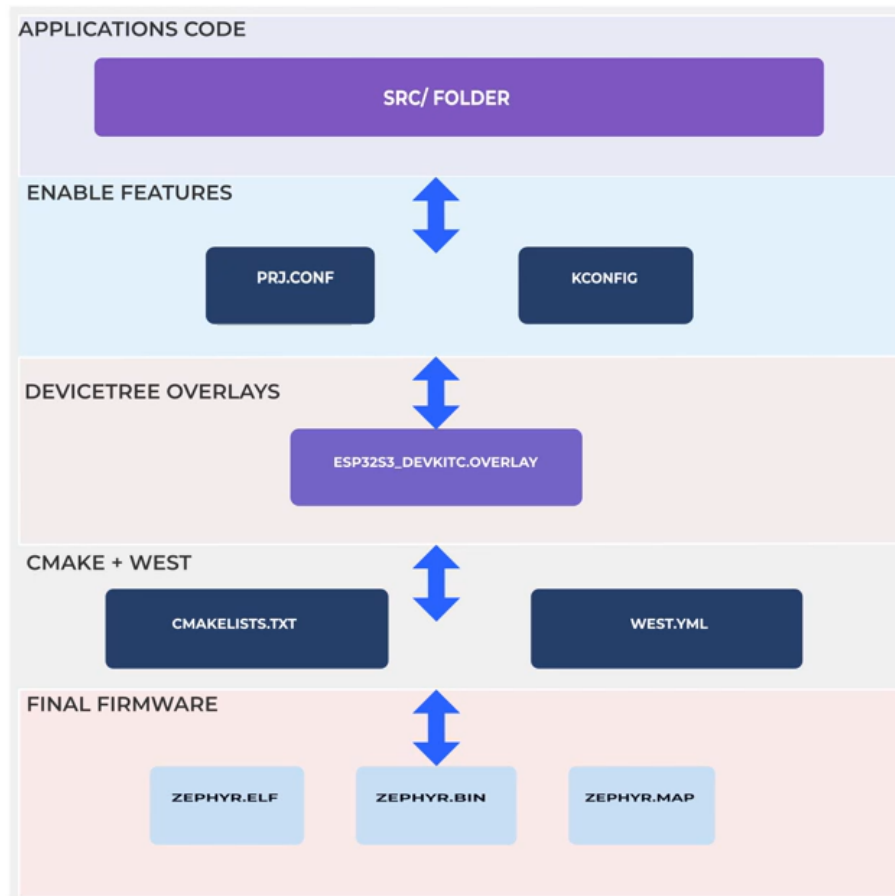


Figure 6.2: Zephyr Project Structure



Zephyr projects have clear app directories plus shared modules.

6.1.9 An Introduction to Kconfig / `prj.conf`

- `prj.conf` sets Kconfig symbols that shape features, drivers, and build options.
- Kconfig enforces dependencies and provides defaults for each board and module.
- Overlay configs (e.g., `boards/*.conf`) fine-tune per-board tweaks.



Kconfig is Zephyr's feature control system for your app.

6.1.10 An Introduction to Devicetree

- Devicetree describes what hardware exists and how it is connected.
- Board and SoC definitions live in `.dts` and `.dtsi` files.
- Applications customize hardware using Devicetree overlays.
- Zephyr generates headers and macros from Devicetree at build time

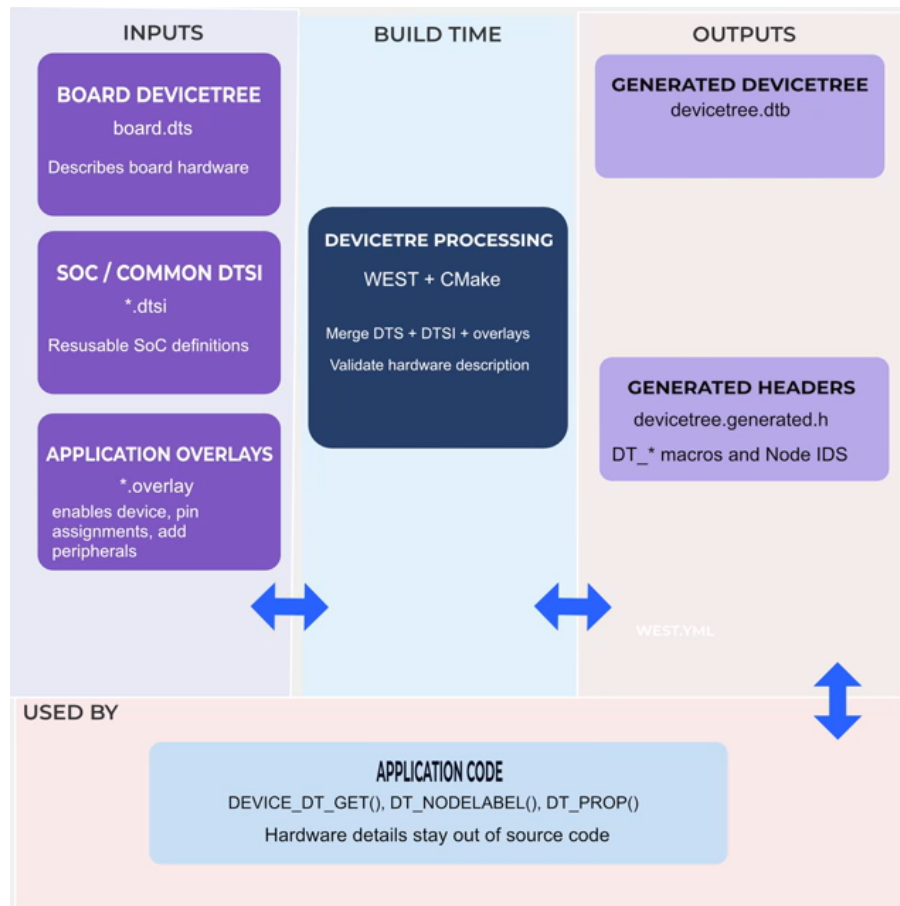


Figure 6.3: Devicetree



In Zephyr the devicetree is the single source of truth for your hardware

6.1.11 West Fundamentals

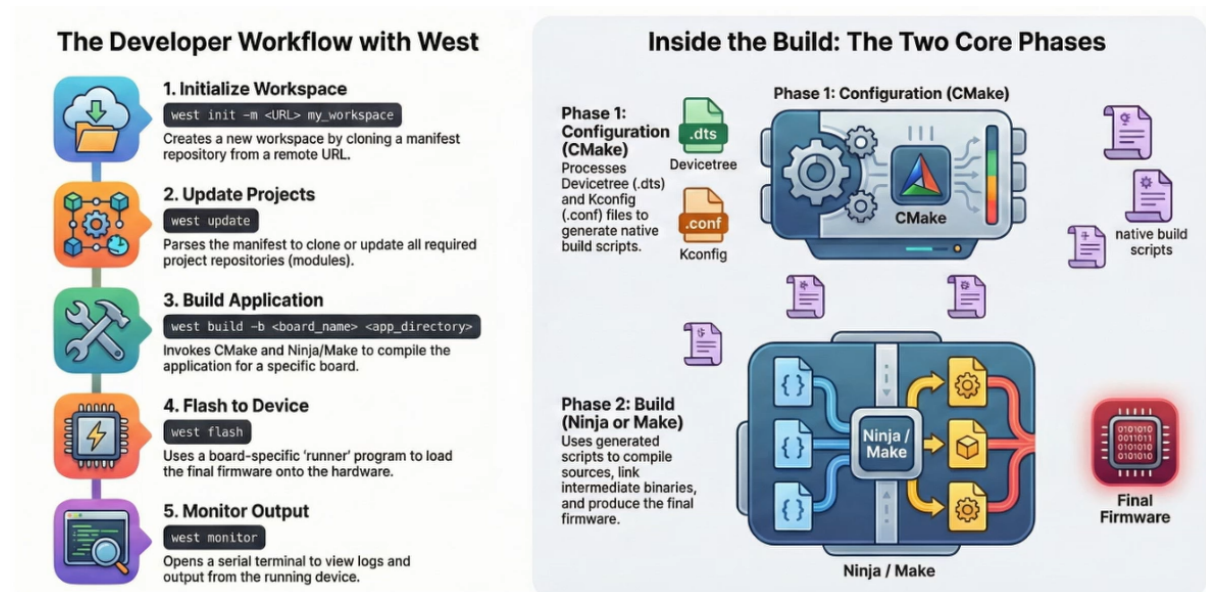


Figure 6.4: Zephyr build process: Command to Firmware

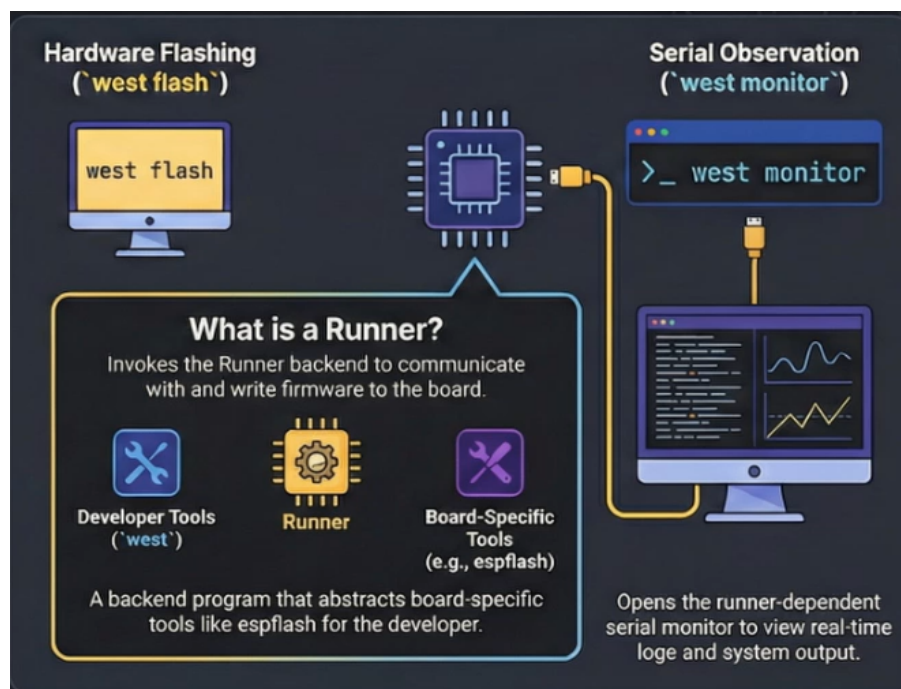


Figure 6.5: West Runners

6.1.12 Zephyr logging

- Zephyr logging provides leveled logs (error to debug) with deferred processing.
- Backends include RTT(Real-Time Transfer), UART and others. We can choose per board and bandwidth needs.
- Configure levels and formatting via `CONFIG_LOG_*` options in `prj.conf`.

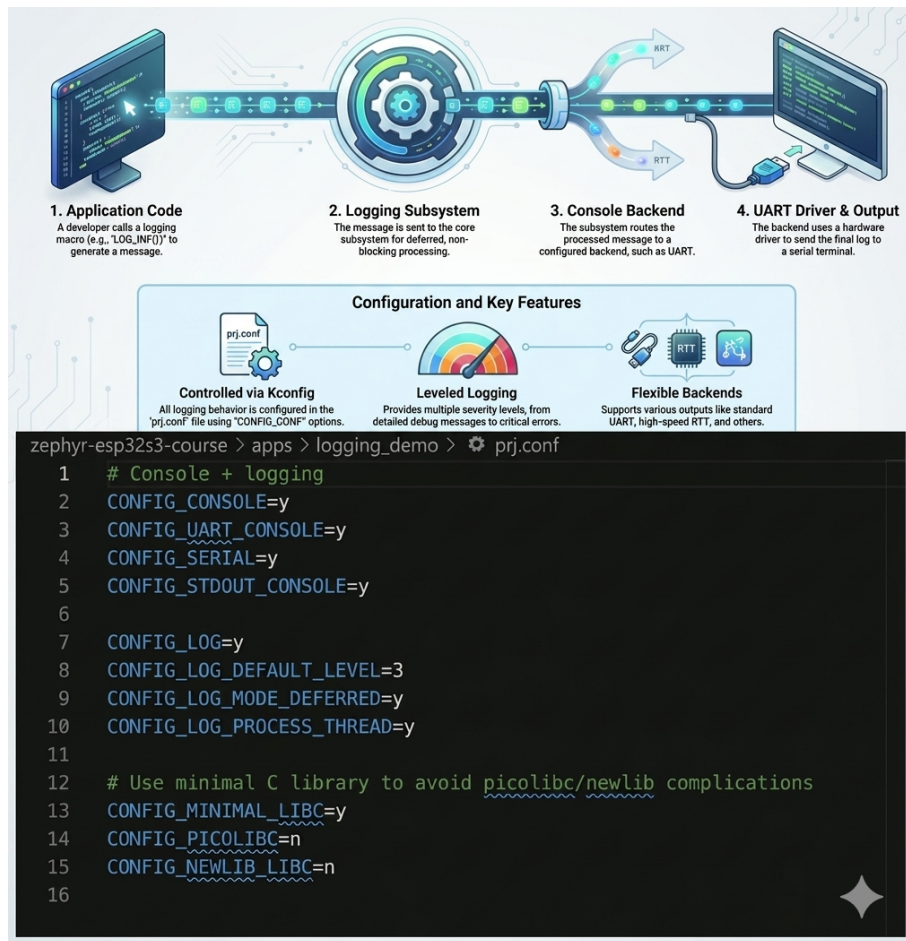


Figure 6.6: Zephyr logging

6.2 Examples

6.2.1 Blinky